Sustavi baza podataka

Predavanja

3. Obavljanje upita

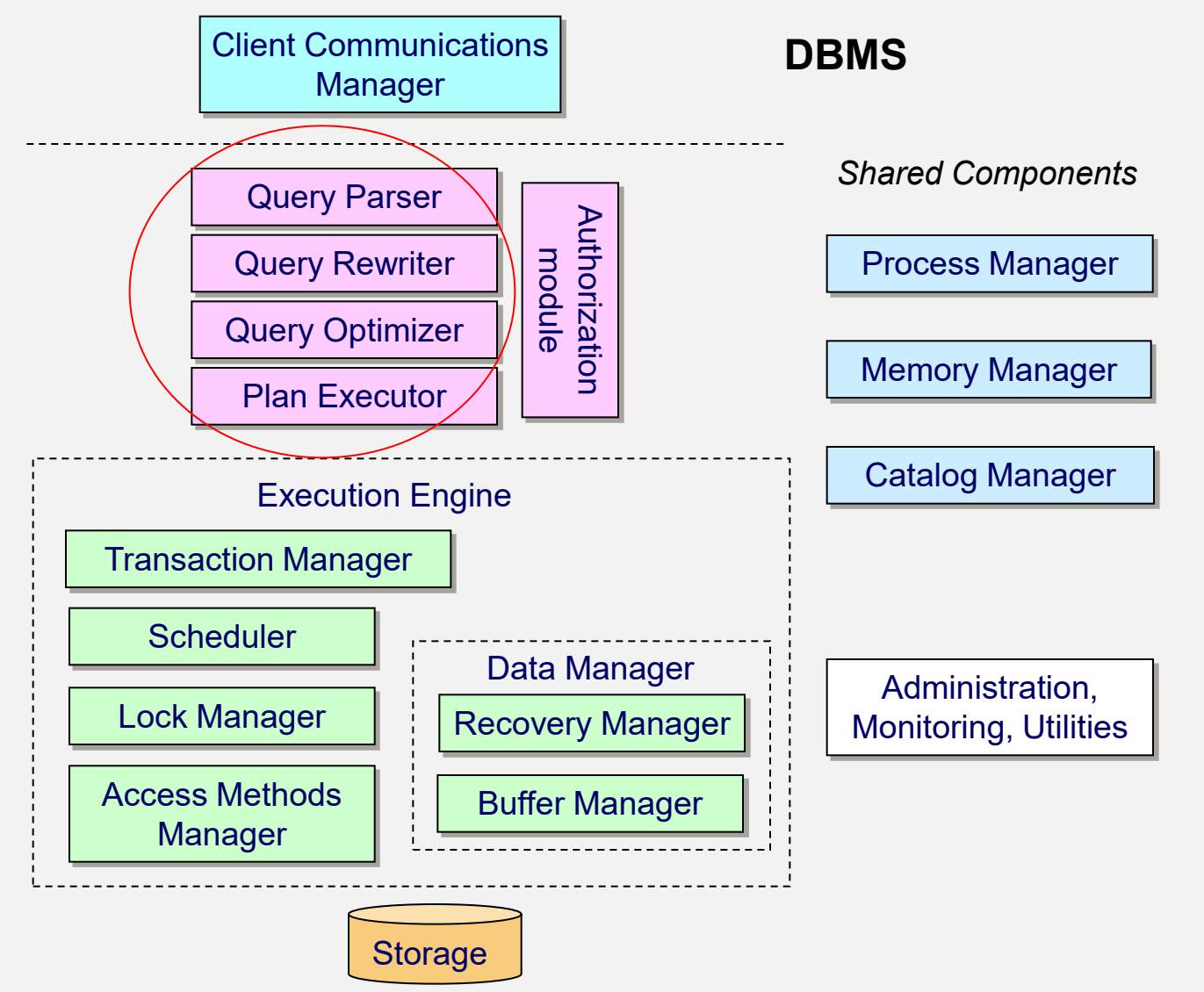
(1. dio)

ožujak 2026.

Obavljanje upita

- obavljanje upita (*query processing*)
  - skup aktivnosti vezanih uz ekstrakciju podataka iz baze podataka
- starije generacije sustava za upravljanje bazama podataka
  - mrežni, hijerarhijski model podataka
  - upitni jezik ugniježđen u proceduralni jezik treće generacije (npr. COBOL, PL/I). Navigacijski upitni jezici: *Network DML*, *Hierarchical DML*
    - odgovornost programera pri odabiru "optimalne" strategije ekstrakcije podataka, programer opisuje postupak za dohvat podataka
- moderni sustavi za upravljanje bazama podataka
  - relacijski i objektno-relacijski modeli podataka
  - SQL je neproceduralni jezik. Korisnik/programer definira *što* rezultat treba sadržavati, a ne postupak kojim se do rezultata treba doći
  - SUBP odabire najprikladniji plan izvršavanja upita

Komponente SUBP-a



Obavljanje upita

Primjer:

```
SELECT DISTINCT imeStud, prezStud
   FROM stud
        JOIN ispit ON stud.mbr = ispit.mbrStud
        JOIN mjesto ON stud.pbrStan = mjesto.pbr
   WHERE nazMjesto = 'Zagreb'
     AND ocjena = 4;
```

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. DISTINCT
6. UNION
7. ORDER BY

$$\pi_{\text{imeStud, prezStud}}(\sigma_{\text{nazMjesto='Zagreb' AND ocjena=4}}((\text{stud} \underset{\text{mbr=mbrStud}}{\triangleright\!\triangleleft} \text{ispit}) \underset{\text{pbrStan=pbr}}{\triangleright\!\triangleleft} \text{mjesto}))$$

```
SELECT DISTINCT imeStud, prezStud
   FROM mjesto CROSS JOIN stud
               CROSS JOIN ispit
   WHERE nazMjesto = 'Zagreb'
     AND ocjena = 4
     AND stud.pbrStan = mjesto.pbr
     AND ispit.mbrStud = stud.mbr;
```

$$\pi_{\text{imeStud, prezStud}}(\sigma_{\text{nazMjesto='Zagreb' AND ocjena=4 AND pbrStan=pbr AND mbrStud=mbr}}(\text{mjesto} \times \text{stud} \times \text{ispit}))$$

SUBP će u oba slučaja obaviti istu operaciju:

$$\pi_{\text{imeStud, prezStud}}(\sigma_{\text{ocjena=4}}(\text{ispit}) \underset{\text{mbr=mbrStud}}{\triangleright\!\triangleleft} (\text{stud} \underset{\text{pbrStan=pbr}}{\triangleright\!\triangleleft} \sigma_{\text{nazMjesto='Zagreb'}}(\text{mjesto})))$$

Obavljanje upita

```
SELECT AVG(ocjena)
   FROM stud
        JOIN ispit ON stud.mbr = ispit.mbrStud
        JOIN mjesto ON stud.pbrStan = mjesto.pbr
   WHERE nazMjesto = 'Zagreb';
```

card(mjesto) = 10 000
card(stud) = 50 000
card(ispit) = 1 000 000
bez indeksa

$$\mathcal{G}_{AVG(ocjena)}(\sigma_{nazMjesto='Zagreb'}(mjesto) \underset{pbr=pbrStan}{\triangleright\!\triangleleft} stud) \underset{mbr=mbrStud}{\triangleright\!\triangleleft} ispit)$$

⟶ 1.8 sec

Eksplicitni zahtjev optimizatoru za promjenom plana izvršavanja istog upita

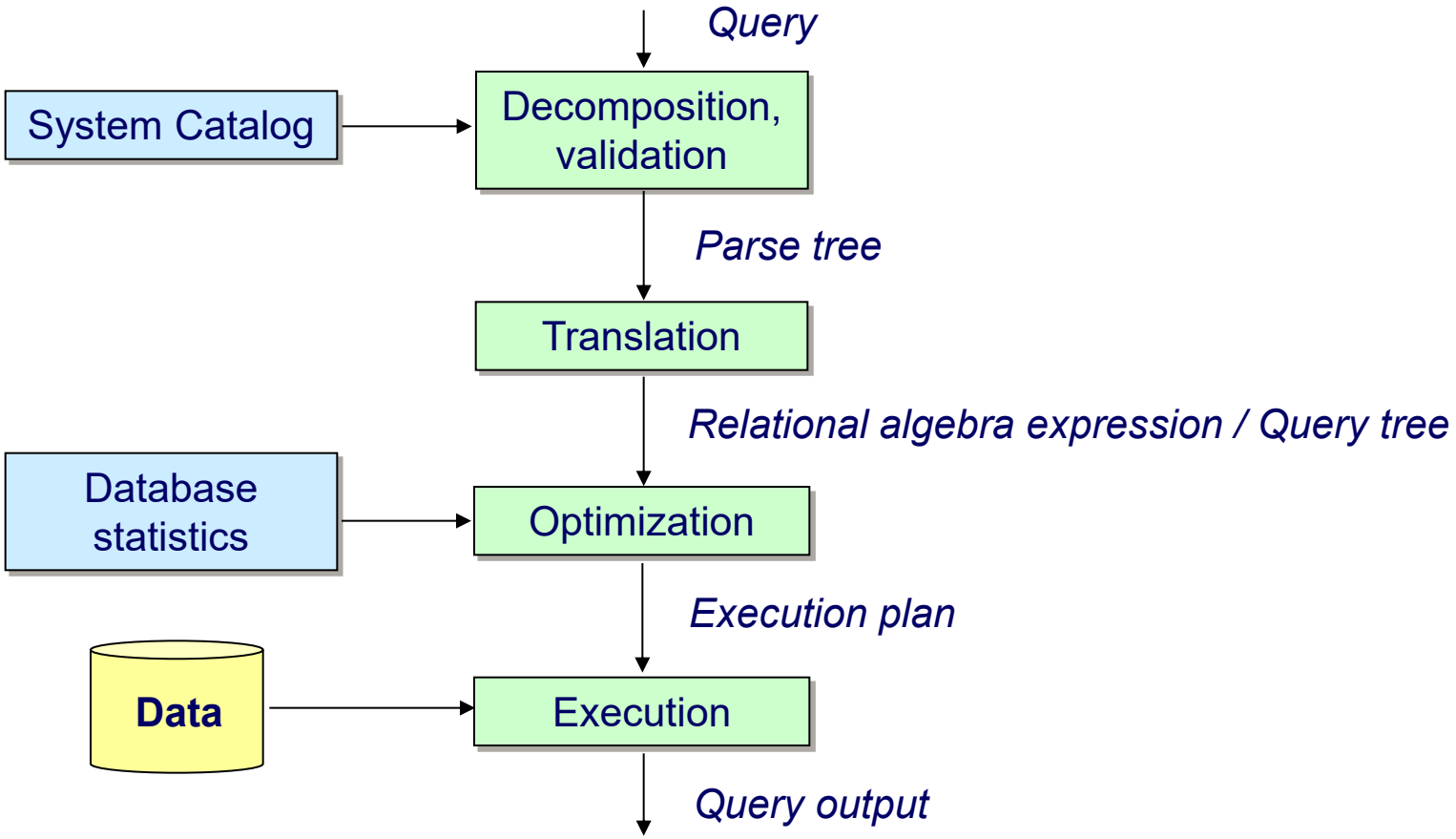
- ORDERED: relacije spajati redoslijedom kojim su navedene u listi FROM

```
SELECT {+ORDERED} AVG(ocjena)
   FROM stud
        JOIN ispit ON stud.mbr = ispit.mbrStud
        JOIN mjesto ON stud.pbrStan = mjesto.pbr
   WHERE nazMjesto = 'Zagreb';
```

$$\mathcal{G}_{AVG(ocjena)}((stud \underset{mbr=mbrStud}{\triangleright\!\triangleleft} ispit) \underset{pbrStan=pbr}{\triangleright\!\triangleleft} \sigma_{nazMjesto='Zagreb'}(mjesto))$$

⟶ 10.5 sec

Obavljanje upita



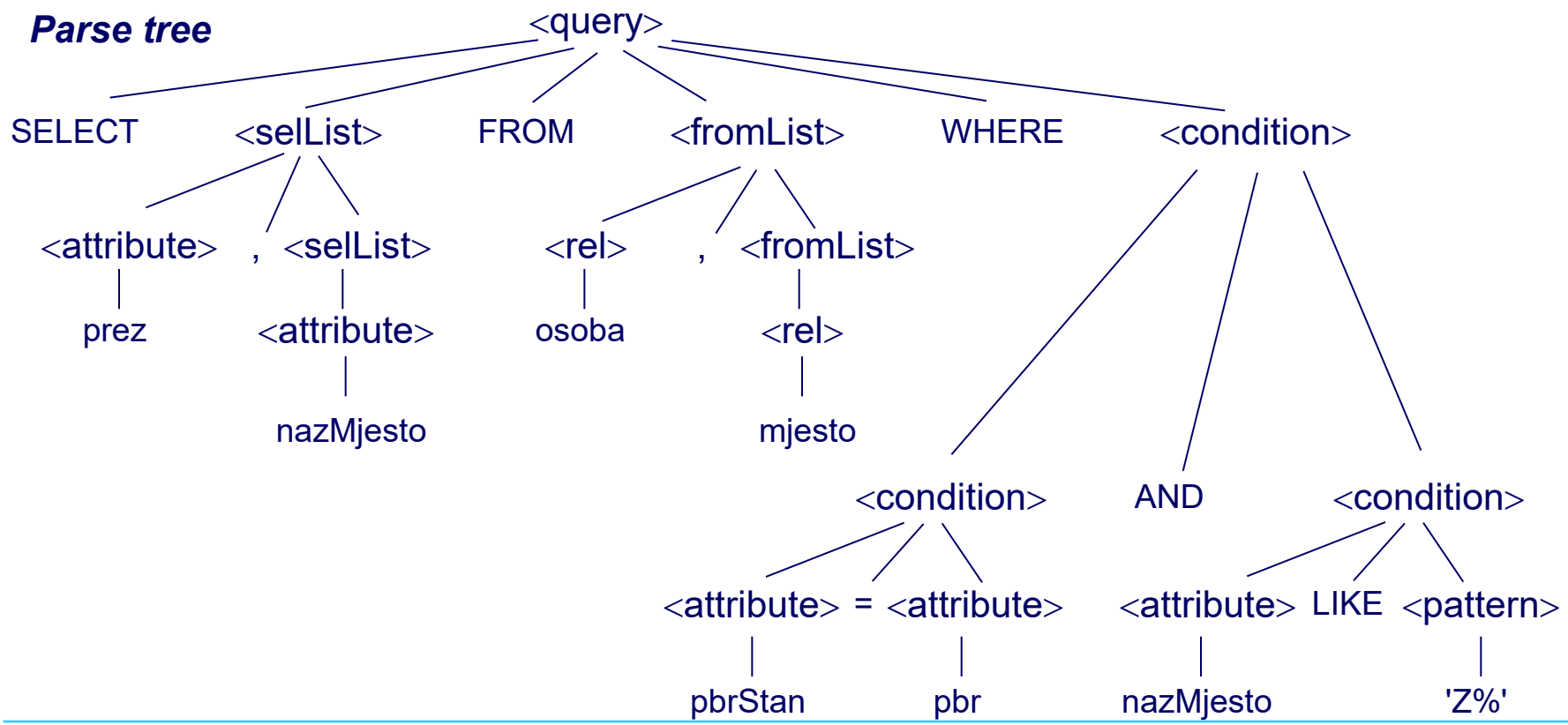
Dekompozicija i validacija

- leksička i sintaktička analiza (tehnike prevođenja programskih jezika)

Primjer:

```
SELECT prez, nazMjesto FROM osoba, mjesto
  WHERE osoba.pbrStan = mjesto.pbr
    AND mjesto.nazMjesto LIKE 'Z%'
```

Parse tree



Dekompozicija i validacija

- obavlja se zamjena virtualne relacije (*view*) stablom (*parse tree*) koje proizlazi iz definicije virtualne relacije u rječniku podataka (modifikacija upita)
  - materijalizirane virt. relacije se promatraju jednako kao temeljne relacije
- obavlja se transformacija podupita
- validacija (semantička analiza):
  - ispitati korištenje naziva relacija. Npr. svaka "relacija" iz FROM liste mora biti relacija opisana u rječniku podataka
  - verificirati ispravno korištenje naziva atributa
    - razriješiti imena atributa (kojoj relaciji pripada koji atribut, postoje li dvosmislenosti)
    - nalazi li se atribut u dosegu (*scope*) relacije spomenute u pripadnoj FROM listi (→Baze podataka→podupiti)
  - provjeriti tipove podataka i primjenjivost operacija nad objektima (npr. množenje cijelog broja i niza znakova)
- pojednostavljivanje i normalizacija (*query rewriter*)
  - npr. predikati za selekciju → konjunktivna normalna forma
  - eliminacija tautologija i kontradikcija

Translacija u inicijalni izraz relacijske algebre

- Inicijalni izraz relacijske algebre nastaje jednostavnom transformacijom stabla parsiranja

- pravilo za transformaciju stabla (*parse tree*) u inicijalni izraz relacijske algebre - za jednostavne upite
  1. načiniti Kartezijev produkt svih relacija navedenih u <fromList>
  2. nad rezultatom prvog koraka obaviti operaciju selekcije σ_F gdje je F uvjet naveden u <condition> po potrebi nadopunjen uvjetima spajanja
  3. nad rezultatom drugog koraka obaviti operaciju projekcije π_L gdje je L lista izraza navedenih u <selList>
     - *bag* verziju operacije, ako nije navedeno DISTINCT

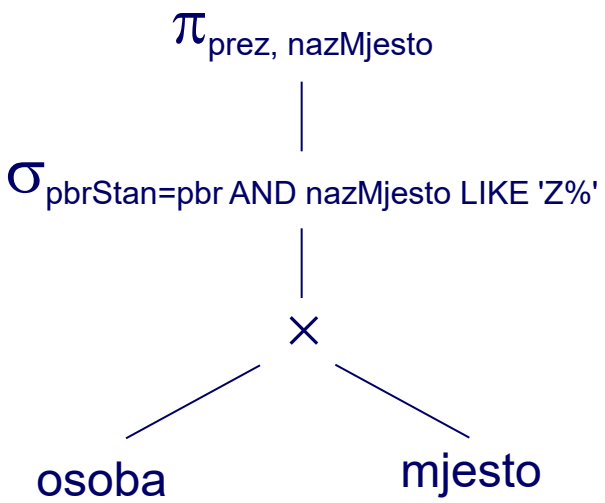
Translacija u inicijalni izraz relacijske algebre

Primjer:

```
SELECT DISTINCT prez, nazMjesto
  FROM osoba
  JOIN mjesto
    ON osoba.pbrStan = mjesto.pbr
  WHERE mjesto.nazMjesto LIKE 'Z%'
```

- inicijalni izraz relacijske algebre:

$$\pi_{\text{prez, nazMjesto}}(\sigma_{\text{pbrStan=pbr AND nazMjesto LIKE 'Z%'}}(\text{osoba} \times \text{mjesto}))$$

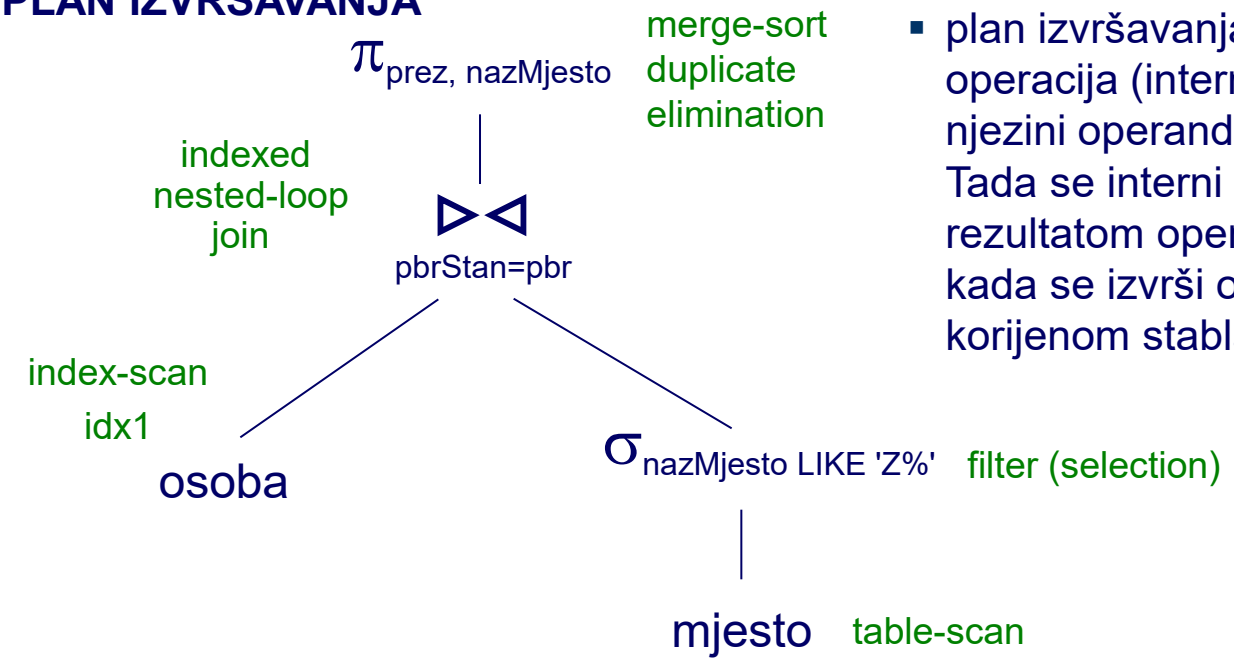


- stablo upita (*query tree*) je način prikaza izraza relacijske algebre
- listovi stabla su relacije, a interni čvorovi su operacije relacijske algebre
- konačni rezultat dobije se obavljanjem operacije u korijenu stabla

Stablo upita → *optimizacija* → Plan izvršavanja upita

- plan izvršavanja upita (*execution plan*, *execution strategy*) obuhvaća
  1. stablo upita: operande i operacije, redoslijed njihovog izvršavanja
  2. fizičke metode izvršavanja operacija relacijske algebre, odnosno fizičke operatore (*physical operators*)
     - *index-scan, table-scan, nested-loop join, merge join, hash join, ...*
  3. način prosljeđivanja međurezultata

PLAN IZVRŠAVANJA



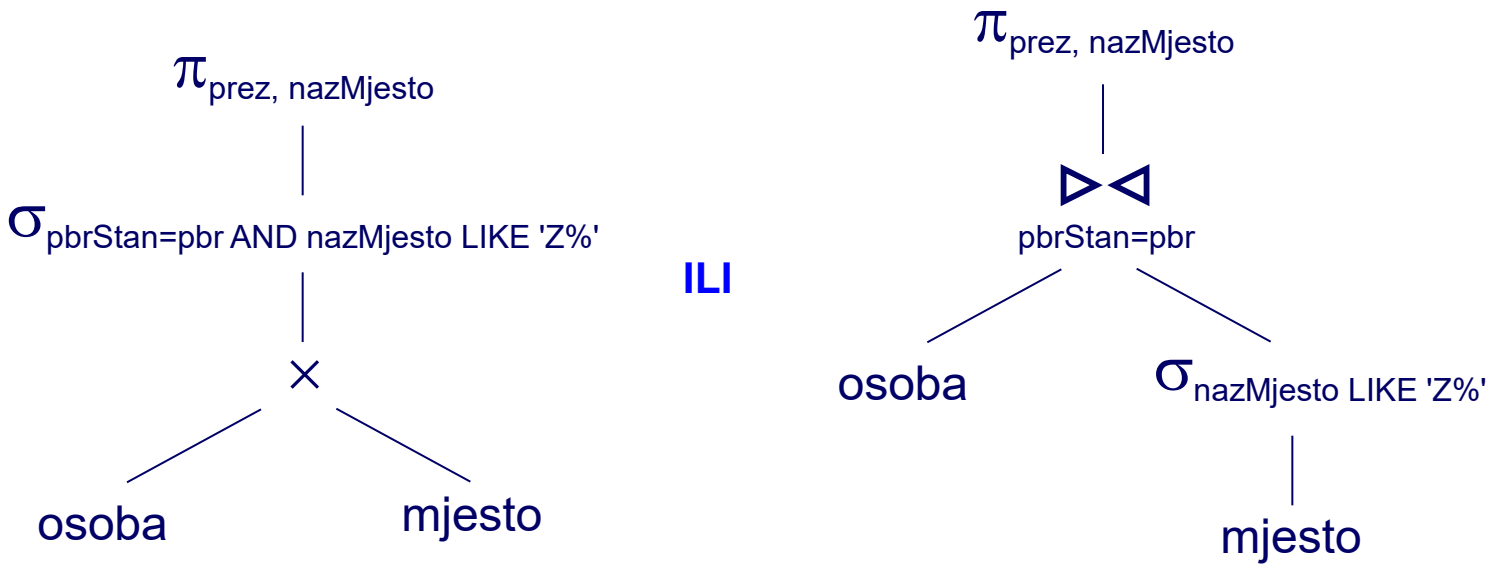
- plan izvršavanja upita se evaluira tako da se operacija (interni čvor) izvršava onda kada njezini operandi (djeca) postanu raspoloživi. Tada se interni čvor zamjenjuje relacijom - rezultatom operacije. Izvršavanje završava kada se izvrši operacija predstavljena korijenom stabla

- na slici, zbog preglednosti, nisu opisani načini prosljeđivanja međurezultata

Plan izvršavanja nije jednoznačno određen upitom

1. **operandi i operacije relacijske algebre, redoslijed njihovog izvršavanja**
   - u općem slučaju se izraz relacijske algebre može zamijeniti ekvivalentnim, alternativnim izrazom relacijske algebre (jednim od mnogih)
     - *dva izraza relacijske algebre su ekvivalentni ako primijenjeni nad svakom instancom baze podataka daju međusobno jednaki rezultat*
   - alternativni izrazi (stabla upita) se određuju na temelju pravila za transformaciju izraza relacijske algebre

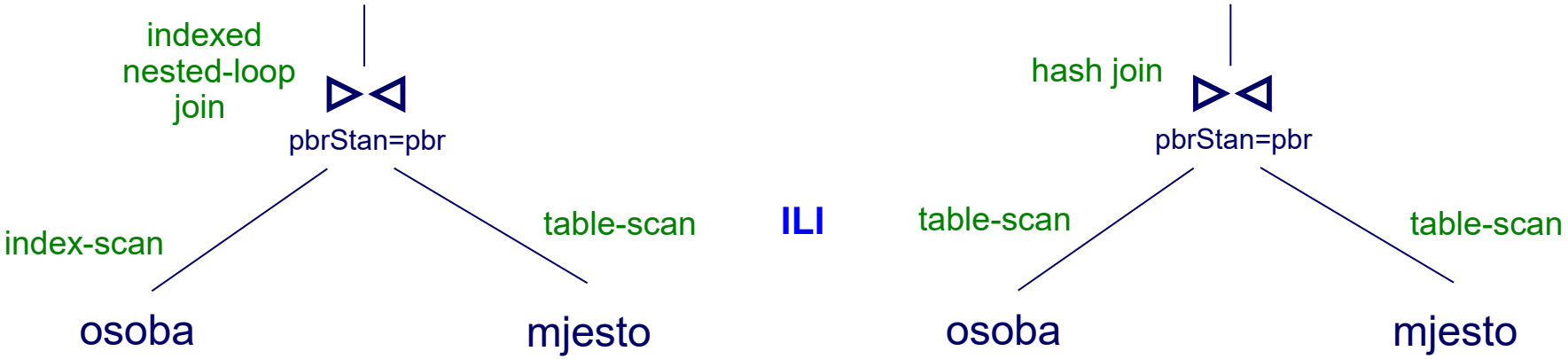
Primjer:



Plan izvršavanja nije jednoznačno određen upitom

2. **fizičke metode izvršavanja algebarskih operacija**
   - SUBP implementira različite metode izvršavanja karakterističnih algebarskih operacija (selekcija, spajanje, projekcija, ...)
   - mogućnost primjene pojedine metode ovisi o konkretno primijenjenoj fizičkoj organizaciji (npr. postoji li odgovarajući indeks u konkretnoj relaciji)
   - razmatraju se alternativne metode izvršavanja algebarskih operacija, njihova primjenjivost i efikasnost

Primjer:



3. **način prosljeđivanja međurezultata** (kasnije)

Optimizacija

$\Rightarrow$ za jedan upit postoji više (potencijalno mnogo) alternativnih planova izvršavanja

- izvršavanjem "lošeg" plana (umjesto "dobrog" plana), utrošak resursa se može povećati za nekoliko redova veličine
- procjenjuju se troškovi za
  - UI operacije (značajno za velike baze podataka)
  - CPU (značajno za male i *in-memory* baze podataka)
  - komunikaciju (značajno za distribuirane SUBP)
- proces odabira *najprikladnijeg* plana izvršavanja naziva se optimizacija upita (*query optimization*)
  - donekle pogrešni termin: odabrani plan nije uvijek optimalan
  - međutim, određivanje optimalnog plana moglo bi biti vremenski prezahtjevno ili neizvedivo zbog nedostatka potrebnih informacija
  - cilj "optimizacije" upita jest odabrati *razumno efikasan* plan izvršavanja

Dinamička i statička optimizacija upita

- *dynamic query optimization*
  - optimizacija se provodi svaki puta kada se pokrene izvršavanje upita
  - dobro: informacije na temelju kojih se provodi optimizacija su ažurne
  - loše: veći utrošak resursa za postupke optimizacije

- *static query optimization*
  - upit se optimizira jednom, generirani plan izvršavanja ("izvršni kôd") se pohranjuje, te se koristi pri pokretanju svakog "sličnog" upita
    - neovisno o SQL sjednici
  - dobro: optimizacija se obavlja relativno rijetko pa se može provesti uz razmatranje većeg broja alternativnih planova
  - loše: podaci na temelju kojih je provedena optimizacija (statistički podaci) se mijenjaju tijekom vremena. U jednom trenutku dobar plan može u promijenjenim okolnostima postati loš

- *hybrid approach*
  - koristi se statička optimizacija, ali sustav ponavlja postupak optimizacije upita kada se statistički podaci značajnije promijene

Fizički operatori

- implementacija i procjena troškova

Implementacija operacija relacijske algebre

- fizički operator je konkretna implementacija (u SUBP-u) algoritama za obavljanje operacija relacijske algebre:
  - selekcija, spajanje, Kartezijev produkt, projekcija
  - grupiranje i agregatne funkcije
  - unija, presjek, razlika
- za obavljanje dodatnih operacija iz SQL jezika:
  - eliminacija duplikata
  - sortiranje
- za obavljanje ostalih fizičkih operacija, npr:
  - čitanje n-torki slijednim čitanjem blokova podataka (*table-scan*)
  - čitanje n-torki uz korištenje indeksa (*index-scan*)
  - čitanje samo vrijednosti ključeva iz indeksa (*key-only index-scan*)

Bag (*multiset*, multiskup) verzije operacija

Podsjetnik
(uzeti u obzir)

- specifičnost SQL-a u odnosu na relacijski model podataka
  - tablica (*table*) u SQL-u nije nužno relacija (skup n-torki), nego multiskup (*multiset*, *bag*) n-torki. Stoga se i skup operacija mora proširiti operacijama koje kao operande koriste multiskupove

"*bag*" verzije operacija unije, presjeka i razlike

- neka su r i s "relacije" u kojima je dopuštena pojava više istih n-torki
- ako se n-torka t u "relaciji" r nalazi *m* puta, u "relaciji" s se nalazi *n* puta, tada
  - u rezultatu r $\cup^B$ s n-torka t se nalazi *m+n* puta — `UNION ALL`
  - u rezultatu r $\cap^B$ s n-torka t se nalazi *min(m, n)* puta — `INTERSECT ALL`
  - u rezultatu r $\setminus^B$ s n-torka t se nalazi *max(0, m-n)* puta — `EXCEPT ALL`

"*bag*" verzija operacija projekcije

- $\pi^B_L(r)$ - izdvajanje vertikalnog podskupa relacije prema listi atributa L (ali bez eliminacije duplikata)

Bag (*multiset*, multiskup) verzije operacija

Podsjetnik
(uzeti u obzir)

Primjer:

r

E	F
1	A
1	A
2	B
2	B
2	B

s

E	F
1	A
1	A
2	B

t

F	G
A	5
A	6
C	7

$r \cup^B s$

E	F
1	A
1	A
1	A
1	A
2	B
2	B
2	B
2	B

$r \cap^B s$

E	F
1	A
1	A
2	B

$r \setminus^B s$

E	F
2	B
2	B

$\pi^B_E(s)$

E
1
1
2

slično i za ostale operacije:

$\sigma^B_{E=1}(s)$

E	F
1	A
1	A

$s \triangleright\!\triangleleft^B t$

E	F	G
1	A	5
1	A	5
1	A	6
1	A	6

Selekcija

- način obavljanja operacije ovisi o uvjetu selekcije (*filter*) i raspoloživim metodama pristupa (*access methods*)
  - *filter* - u ovom području često korišten alternativni pojam za uvjet selekcije

- **jednostavna selekcija** $\sigma_{A > 5}(r)$
  - slijedno čitanje blokova, selekcija linearnom pretragom
  - dohvaćanje blokova preko B-stabla, selekcija uz pomoć indeksa

- **složena selekcija**
  - konjunktivna forma $\sigma_{A > 5 \wedge B = 100}(r)$
  - disjunktivna forma $\sigma_{A > 5 \vee B = 100}(r)$

Jednostavna selekcija

σ_F (r) gdje je F *jednostavna formula* oblika a θ c

- a atribut
- θ je operacija iz skupa { $<$, $\leq$, $>$, $\geq$, $=$ }
- c je konstanta

Mogući načini obavljanja

1. linearna pretraga (*table-scan, sequential scan, linear search, brute force*) za čitanje svih n-torki uz izdvajanje onih koje zadovoljavaju formulu F
2. indeks za dohvat skupa n-torki prema formuli a θ c ako za atribut *a* postoji indeks

Indeks se također može koristiti za formule oblika *range query*: **`a BETWEEN c1 AND c2`**

Ako postoji odgovarajući indeks, ne znači da će se nužno i koristiti.
U nekim uvjetima optimizator će izbjeći korištenje indeksa i radije koristiti linearnu pretragu!

Jednostavna selekcija pomoću indeksa

a) čitanje n-torki pomoću indeksa (B-stabla)
   - *index-scan*
     - indeks se koristi za pronalaženje kazaljki (RID, row-id) na n-torke koje zadovoljavaju uvjet selekcije, a n-torke (tj. dijelovi n-torki čiji se atributi ne nalaze u ključu B-stabla) dohvaćaju se iz blokova s podacima
   - u sustavu SQL Server ta vrsta operacije naziva se *index-seek*, a u planu izvršavanja prikazuje se kao *index-seek+RID lookup*
     - indeks se koristi za pronalaženje pokazivača (RID) na razini lista (*index-seek*), zatim se iz blokova s podacima dohvaćaju n-torke (*RID lookup*)

   - ova vrsta operacije neće se koristiti ako se iz relacije dohvaća relativno veliki broj n-torki, naročito ako je stupanj grupiranja indeksa loš. U takvom slučaju linearna pretraga je prikladnija jer indeks ne smatramo "*isplativim*"
     - stupanj grupiranja indeksa: što su n-torke bolje poredane prema vrijednosti ključa indeksa, stupanj grupiranja je veći (ili bolji)

Jednostavna selekcija pomoću indeksa

b) čitanje samo vrijednosti ključeva iz listova indeksa (B-stabla)
- *key-only index-scan*
  - vrijednosti atributa koje treba pročitati odnose se samo na atribute koji čine ključ B-stabla. To znači da SUBP neće trebati dohvaćati n-torke ili dijelove n-torki iz blokova s podacima
  - nije važno koliko je velik skup n-torki koje se čitaju. Čak i u slučaju čitanja vrijednosti ključeva iz listova za sve n-torke iz relacije (kada se indeks očito ne koristi za svoju primarnu funkciju, a to je filtriranje n-torki prema uvjetu selekcije), operacija je ipak prikladnija od linearnog pretraživanja jer više se isplati čitati sve blokove listova nego sve blokove s podacima (zašto? koliki je broj listova u odnosu na broj blokova s podacima?)
- pojmovi u sustavu SQL Server INFORMATIVNO
  - *index-scan*: ako se čitaju vrijednosti iz listova za sve n-torke iz relacije (ne dohvaćaju se blokovi)
  - *index-seek (*bez spominjanja *+RID lookup)*: ako se indeks koristi za filtriranje prema uvjetu selekcije (dakle, dohvaćaju se vrijednosti iz samo nekih listova, ne dohvaćaju se blokovi)

Jednostavna selekcija pomoću indeksa

Primjeri za a) i b) - *index-scan* i *key-only index-scan*

```
CREATE INDEX idxOso
  ON osoba(sifra);
```

osoba

sifra	ime
1	Pero
2	Ana
3	Ivo
...	...
10000	Marko

10 000

```
SELECT ime FROM osoba WHERE sifra < 10;
```

index-scan *SQL Server: index-seek + "RID lookup"*

```
SELECT ime FROM osoba WHERE sifra > 10;
```

table-scan *zašto ne index-scan?*

```
SELECT sifra FROM osoba WHERE sifra < 10;
```

key-only index-scan *SQL Server: index-seek (bez "+RID lookup")*

```
SELECT sifra FROM osoba WHERE sifra > 10;
```

key-only index-scan *SQL Server: index-seek (bez "+RID lookup")*

```
SELECT sifra FROM osoba;
```

key-only index-scan *SQL Server: index-scan*

Nazivi fizičkih operatora iz sustava SQL Server navedeni su samo informativno.

Jednostavna selekcija pomoću indeksa

c) (SQL Server) čitanje n-torki iz listova *clustered* indeksa
- *index-seek (clustered)*
- *index-scan (clustered)*

Primjeri:

```
CREATE CLUSTERED INDEX idxOso
  ON osoba(sifra);
```

```
SELECT ime FROM osoba WHERE sifra < 10;
```

SQL Server: clustered index-seek

```
SELECT ime FROM osoba WHERE sifra > 10;
```

SQL Server: clustered index-seek

```
SELECT ime FROM osoba;
```

SQL Server: clustered index-scan

osoba

sifra	ime
1	Pero
2	Ana
3	Ivo
...	...
10000	Marko

10 000

zašto se ovaj indeks koristi čak i onda kada se dohvaća veliki broj n-torki ili čak i sve n-torke?

Složena selekcija

Konjunktivna forma

- Uvjet selekcije je oblika $F_1 \wedge F_2 \wedge F_3$ ..., gdje su F_i *jednostavne formule*

1. konjunktivna selekcija korištenjem složenog (kompozitnog) indeksa
   - ako se u konjunktivnom obliku formule s relacijskim operatorima usporedbe ($<$, $\leq$, $>$, $\geq$, $=$) koriste atributi za koje postoji složeni indeks, npr.
   - ako postoji index **`idx(a,b)`**, može se iskoristiti za predikat:
     - **`a = 7 AND b > 'P'`**
2. konjunktivna selekcija korištenjem jednog indeksa
   - ako se za neki F_i može (i isplati) primijeniti indeks, tada se pomoću indeksa dohvate n-torke koje zadovoljavaju F_i, a nad svakom tako dohvaćenom n-torkom primjenjuje se *filter* sastavljen od preostalih jednostavnih formula, $F_1 \wedge F_2$ ...
   - ako su na raspolaganju indeksi koji se mogu koristiti za više od jedne formule F_i, odabire se onaj F_i (i koristi pripadajući indeks) kojim će se dohvatiti najmanji broj n-torki ili onaj koji će koristiti indeks s najboljim stupnjem grupiranja - razmislite o čemu to ovisi!
3. linearna pretraga

Složena selekcija

Konjunktivna forma

Primjer:

```
SELECT * FROM ispit
  WHERE ocjena = 3
    AND sifPred = 201
    AND datIspit = TODAY
```

- broj n-torki u relaciji 100000
- broj različitih predmeta 500
- broj različitih ocjena 5
- broj različitih datuma 50

Ako bi postojao samo jedan od dolje navedenih indeksa, bilo bi moguće npr.

- koristiti kompozitni indeks za (**`datIspit, ocjena, sifPred`**)
- koristiti kompozitni indeks za (**`sifpred, datIspit`**) (+ filter za ostale uvjete)
- koristiti kompozitni indeks (**`sifPred, ocjena`**) (+ filter za ostale uvjete)
- koristiti indeks za **`sifPred`** (+ filter za ostale uvjete)
- koristiti (ali pitanje je isplati li se! Zašto?) indeks za **`ocjena`** (+ filter za ostale uvjete)

- međutim, ako postoji indeks za **`sifpred`**, indeks za **`ocjena`** i indeks za **`datIspit`**, a svi indeksi imaju jednako loš stupanj grupiranja, koristi se indeks za **`sifPred`** (+ filter za ostale uvjete)
  - zašto se koristi samo jedan indeks i zašto baš taj?

Složena selekcija

Disjunktivna forma (disjunktivna selekcija)

- Uvjet selekcije je oblika $F_1 \vee F_2 \vee F_3$ ..., gdje su F_i *jednostavne formule*
  - samo u slučaju ako postoji (vrlo isplativi) indeks za svaki F_i, zasebno se dohvaća po jedan skup identifikatora n-torki (RID, row-id) za svaki uvjet F_i uz korištenje pripadnog indeksa. Rezultat se dobije dohvatom n-torki putem identifikatora iz unije dohvaćenih skupova identifikatora
    - razmisliti: zašto za primjenu ove metode mora postojati odgovarajući indeks za sve F_i
  - inače, linearna pretraga

Složena selekcija

Disjunktivna forma (disjunktivna selekcija)

Primjer:

```
SELECT * FROM ispit
  WHERE datIspit = TODAY
     OR sifPred = 201
```

- ako postoji samo kompozitni indeks (**sifPred** , **datIspit)** , koristi se linearna pretraga
- ako postoji samo indeks za **sifPred** ili samo indeks za **datIspit**, linearna pretraga!
- ako postoji indeks za **sifpred** i indeks za **datIspit**, dohvaća se skup identifikatora n-torki za koje je **datIspit=TODAY**, skup identifikatora n-torki za koje je **sifPred=201**, nakon čega se obavlja unija tih skupova, te se iz blokova s podacima dohvaćaju sve n-torke s RID-ovima iz dobivene unije
  - ali i u ovakvim slučajevima, ako se očekuje veliki broj n-torki koje zadovoljavaju jednu ili više jednostavnih formula, optimizator će ipak odabrati linearnu pretragu

Statistički podaci iz rječnika podataka

- B(r) - broj blokova relacije r
- N(r) - broj n-torki relacije r
- d(idx) - dubina B-stabla za indeks idx
- V(A, r) - broj različitih vrijednosti atributa A u relaciji r
  - V(A, r) = $\mathcal{G}_{\text{COUNT(*)}}(\pi_{\text{A}}(\text{ r }))$
  - npr. V(ocjena, ispit) = $\mathcal{G}_{\text{COUNT(*)}}(\pi_{\text{ocjena}}(\text{ ispit }))$ = 5
- ostali statistički podaci (na sljedećem predavanju)

- SUBP ne ažurira statističke podatke pri svakoj promjeni podataka, već
  - u unaprijed propisanim vremenima (npr. u vremenu smanjenog opterećenja sustava), reakcijom na neki događaj u sustavu (unos veće količine podataka), itd.
  - odlukom administratora sustava, npr. naredbom oblika

```
UPDATE STATISTICS FOR TABLE r
```

Selekcija - procjena troškova

- procjena troška: procjenjuje se broj U/I operacija koje će biti potrebno obaviti u najlošijem slučaju
- ako nema indeksa za ključ pretrage, moguće je jedino linearnom pretragom: B(r)
- pomoću indeksa *idx* (B^+-stabla)
  - jedna n-torka (preko primarnog ili alternativnog ključa relacije): d(idx) + 1
  - više n-torki: d(idx) + broj listova + broj blokova u kojima se nalaze tražene n-torke
    - indeks s dobrim *stupnjem grupiranja* (npr. Informix *clustered index)* obično nije problem, osim ako se dohvaća relativno veliki broj n-torki, pa se tada ne koristi (više se isplati linearna pretraga)
    - indeks s lošim stupnjem grupiranja, *non-clustered index*: vrijednosti u listovima indeksa jesu sortirane, ali tražene n-torke mogu biti više ili manje raspršene među blokovima s podacima. Osim ako se ne radi o relativno malom broju n-torki koje se dohvaćaju, trošak lako može postati i veći nego trošak linearne pretrage pa se indeks tada ne koristi (više se isplati linearna pretraga)

Za svaki indeks SUBP u rječniku podataka može voditi podatak o "stupnju grupiranja" (*degree of clustering*) koji govori u kojoj mjeri fizički poredak n-torki u blokovima s podacima odgovara poretku vrijednosti ključeva u dotičnom indeksu.

Što se može reći o stupnju grupiranja *clustered* indeksa u sustavu SQL Server?

Primjeri za *clustered* indeks (IBM Informix)

```
CREATE INDEX idxOso ON osoba (sifra);
```

Prema slici se može zaključiti da je stupanj grupiranja vrlo nizak:

```
SELECT ime FROM osoba WHERE sifra < 500;
```

table-scan

osoba

sifra	ime
354	Jura
2	Tomo
8753	Marija
...	...
14	Jana

10 000

```
DROP INDEX idxOso;
CREATE CLUSTER INDEX idxOso ON osoba (sifra);
```

```
SELECT ime FROM osoba WHERE sifra < 500;
```

index-scan

```
SELECT ime FROM osoba WHERE sifra > 10;
```

table-scan

Sortiranje

```
SELECT * FROM osoba
  ORDER BY prezime
```

- sortiranje je jedna od temeljnih fizičkih operacija
  - zbog ORDER BY u SQL naredbi
  - zbog fizičkih operatora koji zahtijevaju da argumenti budu sortirani (vidjeti kasnije npr. *spajanje uparivanjem sortiranih relacija*)

- relacija r(R) i lista atributa L $\subseteq$ R prema kojima treba obaviti sortiranje
  - ako postoji odgovarajući indeks (koji sadrži L u *odgovarajućem* poretku)
    - *key-only index-scan* je prikladan ako ključ indeksa sadrži sve atribute navedene u *select-listi*, neovisno o stupnju grupiranja (za vježbu odgovorite zašto)
    - *index-scan* je prikladan samo ako je stupanj grupiranja jako dobar (za vježbu odgovorite zašto)
  - ako ne postoji odgovarajući indeks ili ga se ne isplati primijeniti, postoje dva scenarija
    - ako n-torke koje treba sortirati stanu u primarnu memoriju koja je na raspolaganju korisničkoj sjednici, tada *table-scan* + sortiranje u primarnoj memoriji
    - inače, vanjsko sortiranje

Vanjsko sortiranje

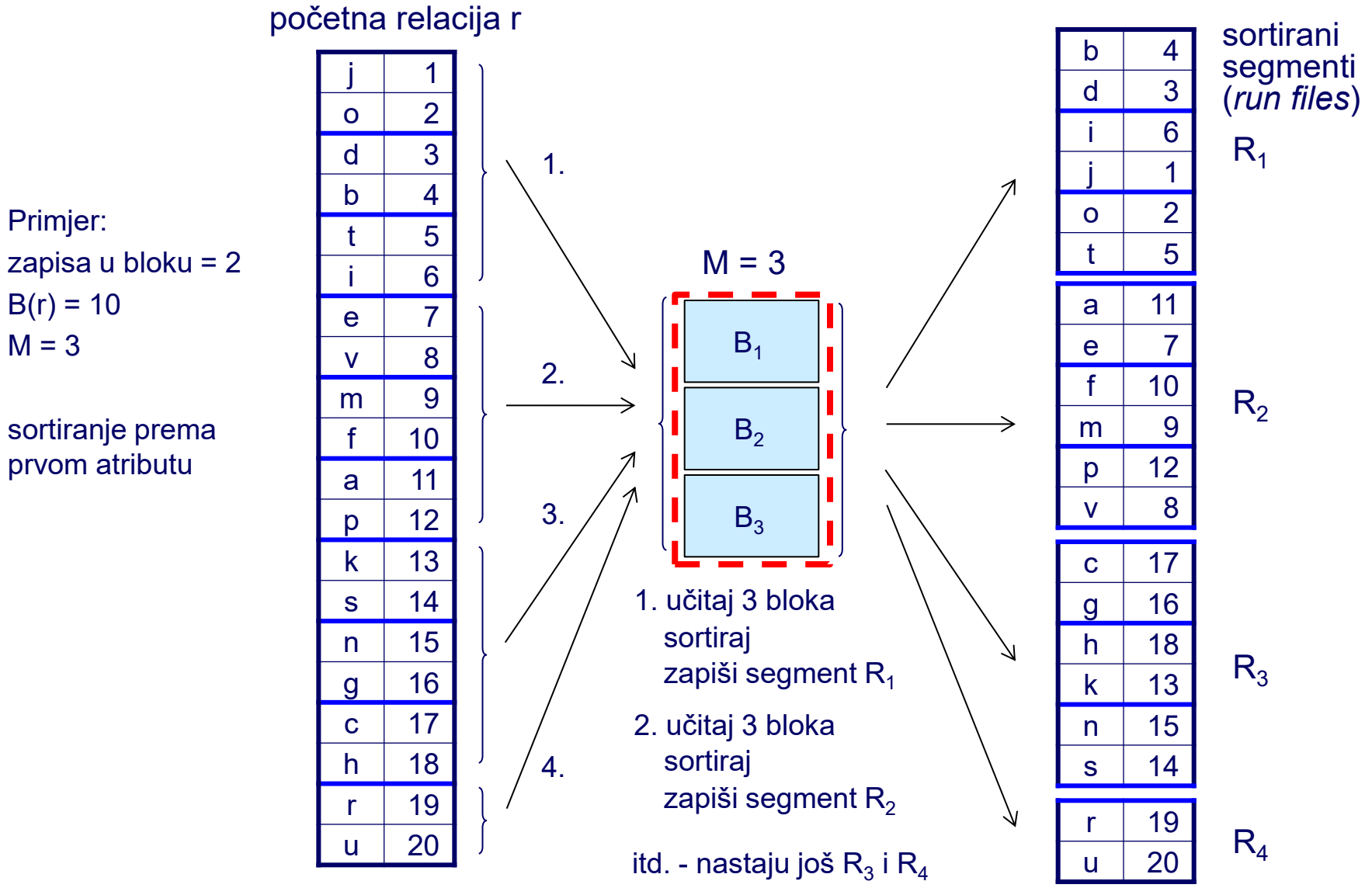
- koristi se za sortiranje relacija koje se ne mogu odjednom smjestiti u segment primarne memorije koji korisnička sjednica ima na raspolaganju
- najpoznatiji i najčešći oblik vanjskog sortiranja je vanjsko sortiranje s uparivanjem: *external sort-merge*
- obavlja se u dvije faze
  - faza sortiranja
  - faza uparivanja

External sort-merge - 1. faza - *sorting phase*

- B(r) - broj blokova relacije r
- M - raspoloživi broj blokova primarne memorije
- relacija r se dijeli na N segmenata veličine M, $N = \lceil B(r) / M \rceil$
  - svaki segment (*run file*) se sortira zasebno i zapisuje u postojanu memoriju

```
i = 0
repeat
    read M blocks of r or the rest of the r
    sort the in-memory part
    write the sorted data to run file R_i
    i = i + 1
until the end of the r
```

External sort-merge - 1. faza - *sorting phase*

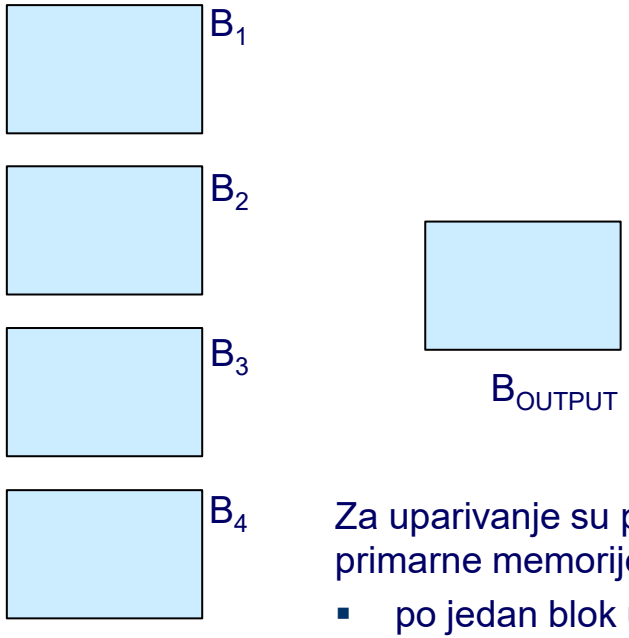


External sort-merge - 2. faza - merging phase

Kada bi za fazu uparivanja na raspolaganju bio dovoljan broj blokova primarne memorije

R_1	b	4
	d	3
	i	6
	j	1
	o	2
	t	5
R_2	a	11
	e	7
	f	10
	m	9
	p	12
	v	8
R_3	c	17
	g	16
	h	18
	k	13
	n	15
	s	14
R_4	r	19
	u	20

Samo radi početnog objašnjenja *uparivanja sortiranih segmenata*, pretpostavit će se da je u fazi uparivanja na raspolaganju dovoljan broj blokova primarne memorije

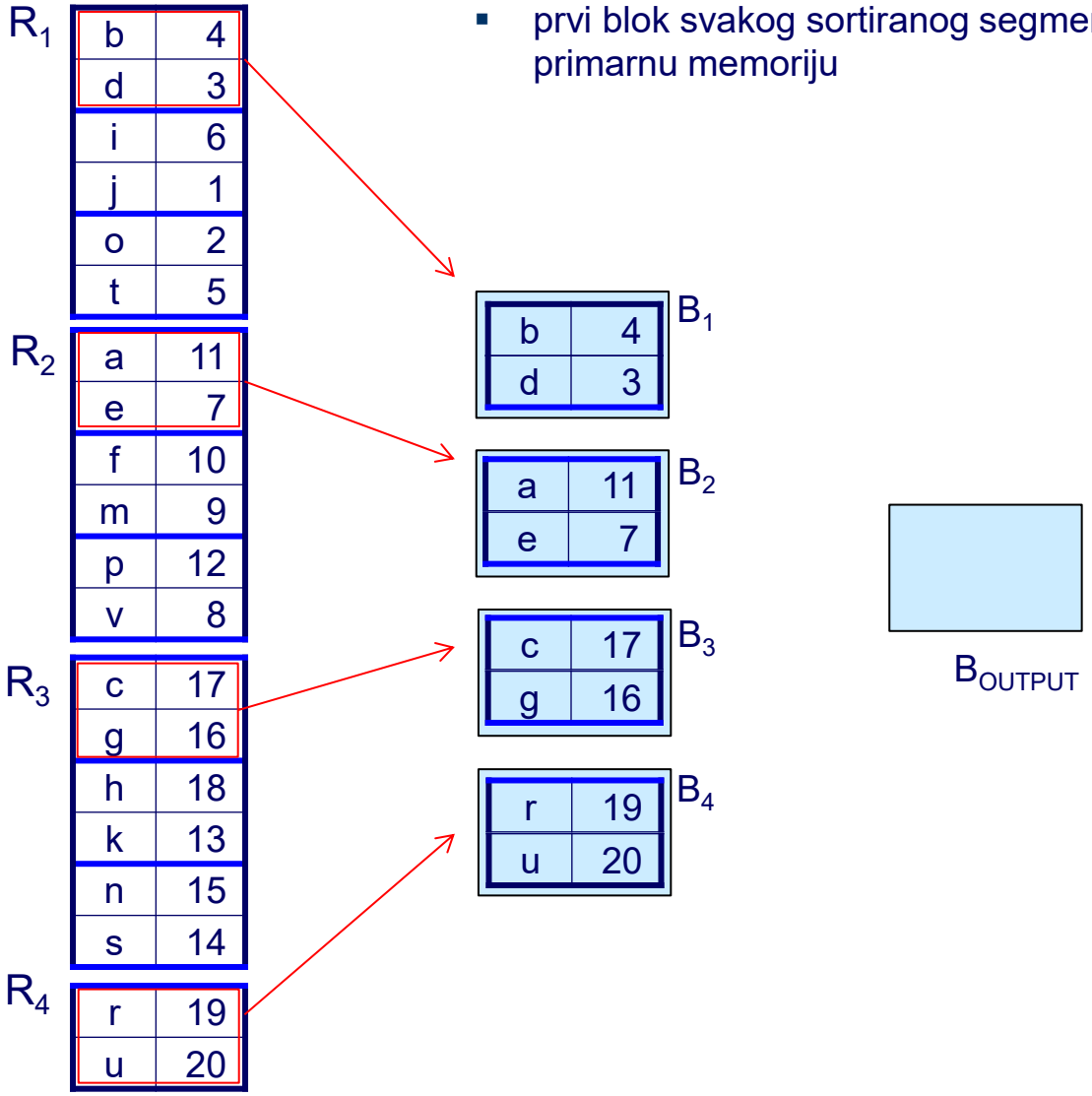


Za uparivanje su potrebni blokovi primarne memorije

- po jedan blok ulaznog međuspremnika (B_1 - B_4) za svaki sortirani segment
- jedan blok B_{OUTPUT} za izlazni međuspremnik

External sort-merge - 2. faza - merging phase

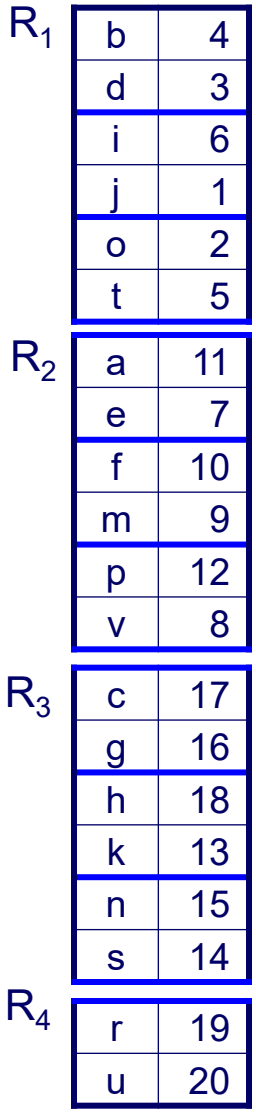
Kada bi za fazu uparivanja na raspolaganju bio dovoljan broj blokova primarne memorije



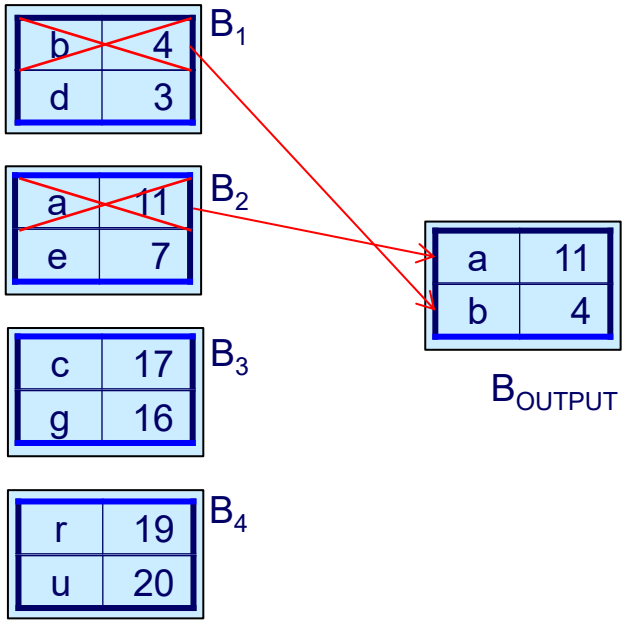
- prvi blok svakog sortiranog segmenta prebaciti u primarnu memoriju

External sort-merge - 2. faza - merging phase

Kada bi za fazu uparivanja na raspolaganju bio dovoljan broj blokova primarne memorije

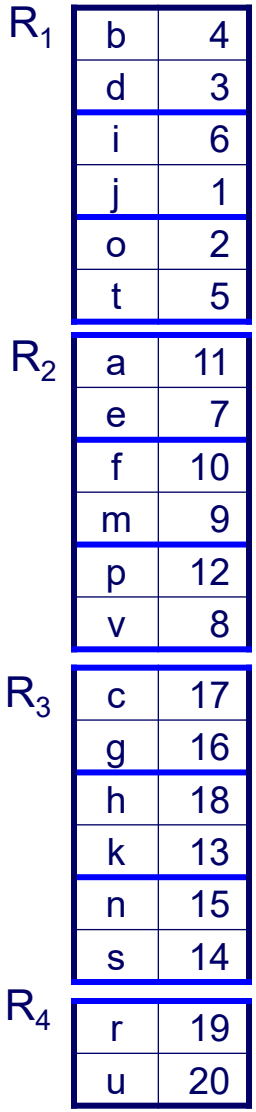


- n-torke iz B_1 - B_4 kopirati u B_{OUTPUT} (redom prema vrijednosti prvog atributa)

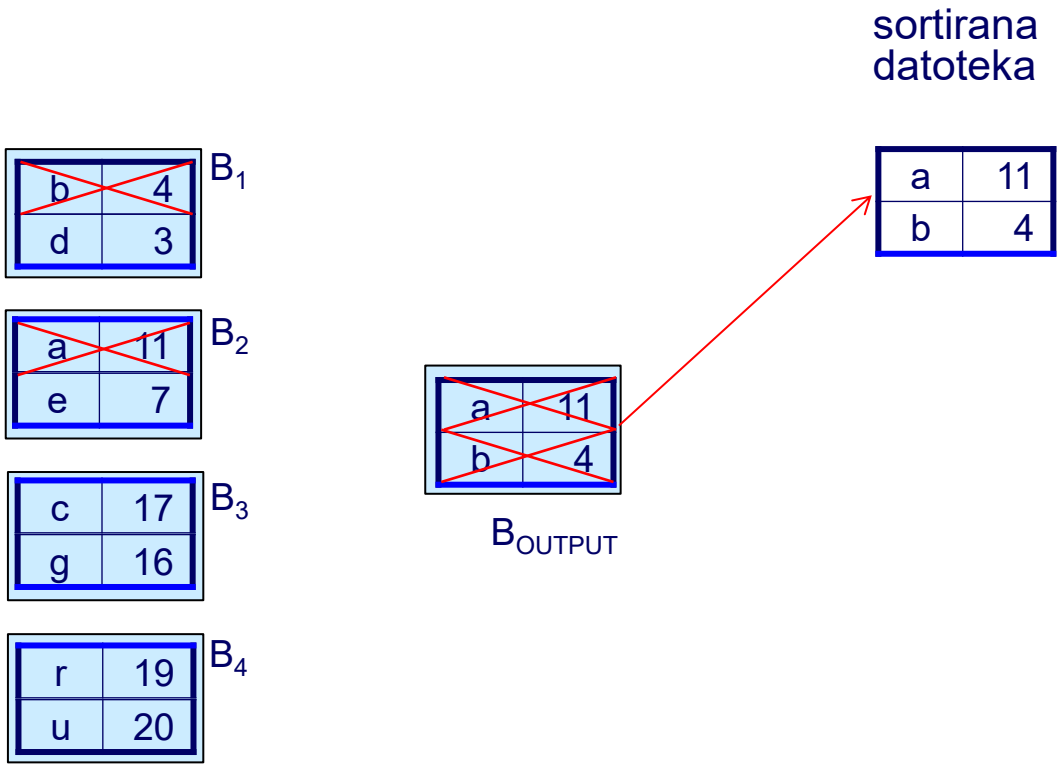


External sort-merge - 2. faza - merging phase

Kada bi za fazu uparivanja na raspolaganju bio dovoljan broj blokova primarne memorije



- kada se izlazni međuspremnik popuni, sadržaj upisati u izlaznu datoteku (koja će na kraju biti potpuno sortirana)

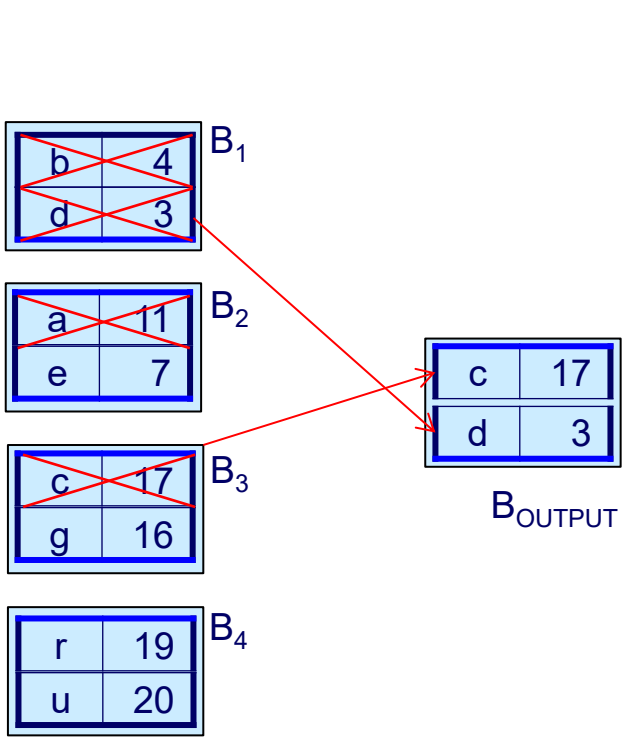


External sort-merge - 2. faza - merging phase

Kada bi za fazu uparivanja na raspolaganju bio dovoljan broj blokova primarne memorije

R_1	b	4
	d	3
	i	6
	j	1
	o	2
	t	5
R_2	a	11
	e	7
	f	10
	m	9
	p	12
	v	8
R_3	c	17
	g	16
	h	18
	k	13
	n	15
	s	14
R_4	r	19
	u	20

- nastaviti popunjavati izlazni međuspremnik n-torkama iz blokova B_1 - B_4 (redom prema vrijednosti prvog atributa)



sortirana datoteka

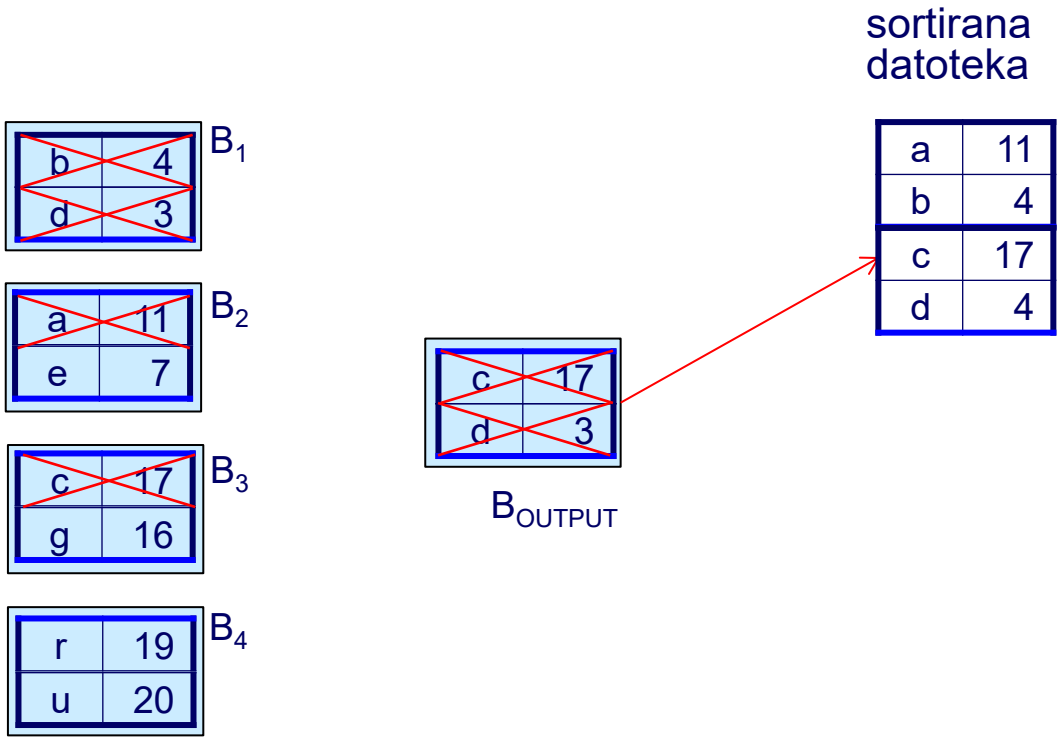
a	11
b	4

External sort-merge - 2. faza - merging phase

Kada bi za fazu uparivanja na raspolaganju bio dovoljan broj blokova primarne memorije

R_1	b	4
	d	3
	i	6
	j	1
	o	2
	t	5
R_2	a	11
	e	7
	f	10
	m	9
	p	12
	v	8
R_3	c	17
	g	16
	h	18
	k	13
	n	15
	s	14
R_4	r	19
	u	20

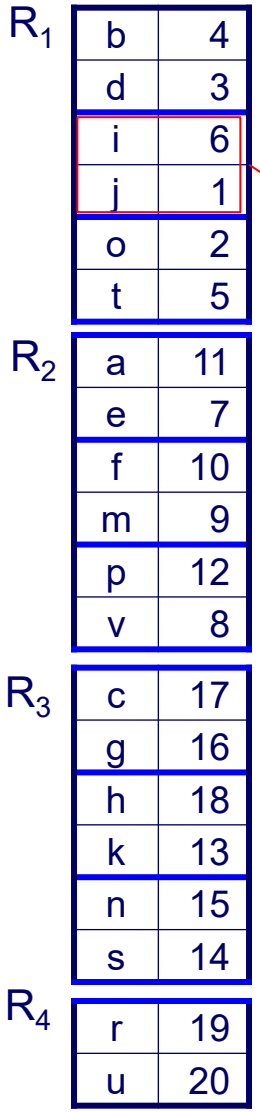
- ponovo, kada se izlazni međuspremnik popuni, sadržaj upisati u izlaznu datoteku



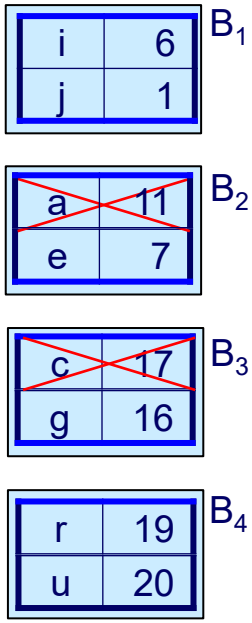
- sve n-torke iz bloka B1 su iskorištene. Što sada?

External sort-merge - 2. faza - merging phase

Kada bi za fazu uparivanja na raspolaganju bio dovoljan broj blokova primarne memorije



- kada se sve n-torke iz nekog od blokova B_1 - B_4 iskoriste, u njega se učitava sljedeći blok iz pripadnog sortiranog segmenta i postupak se nastavlja ...



sortirana datoteka

a	11
b	4
c	17
d	3

B_{OUTPUT}

Međutim, što ako se za fazu uparivanja mogu koristiti samo ukupno 3 bloka, kako je pretpostavljeno na početku primjera?

Tada se segmenti ne uparuju odjednom, nego u grupama. Veličina grupe ovisi o količini raspoložive memorije.

External sort-merge - 2. faza - *merging phase*

- odjednom se uparuju zapisi iz d_m segmenata (d_m je *degree of merging* ili stupanj uparivanja)
  - $d_m = \min(N, M-1)$

- ako je $N < M$, tada je $d_m = N$, te je za uparivanje dovoljan ukupno jedan korak
  - za fazu uparivanja ima dovoljno primarne memorije s obzirom na broj segmenata
  - u stvarnosti, to je najčešći slučaj
- ako je $N \geq M$, uparivanje je potrebno provesti u više koraka
  - za fazu uparivanja nema dovoljno primarne memorije s obzirom na broj segmenata
  - uparuje se po M-1 segmenata. Time se broj sortiranih segmenata u svakom koraku reducira za faktor M-1
  - postupak se ponavlja dok god je broj sortiranih segmenata veći od 1
  - u stvarnosti se, osim u ekstremnim slučajevima, ne očekuje više od ukupno dva koraka uparivanja

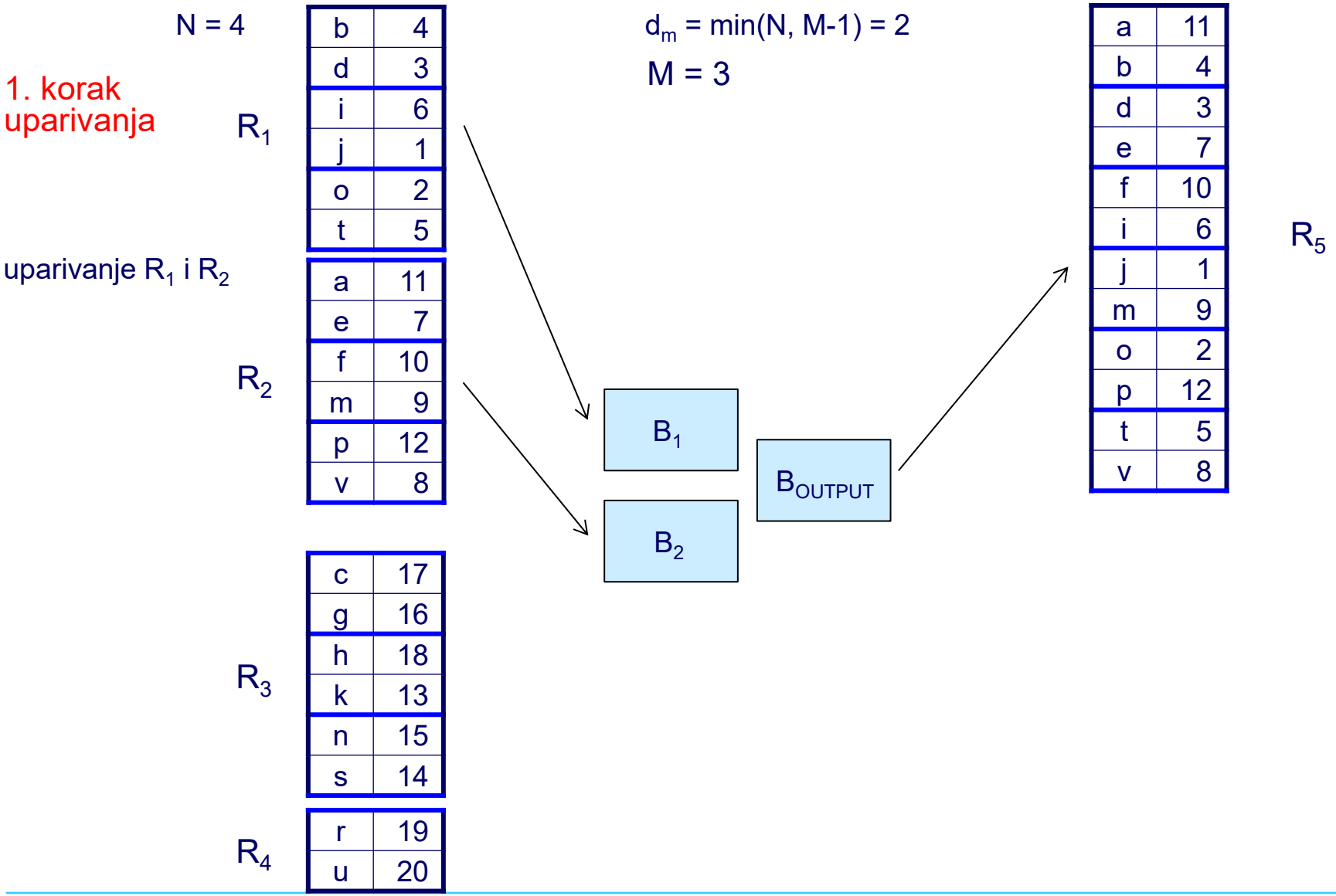
External sort-merge - 2. faza - *merging phase*

- u primarnoj memoriji alocirati po jedan ulazni blok za svaki od segmenata + jedan izlazni blok za (*buffered*!) zapisivanje rezultata, ukupno ne više od M blokova

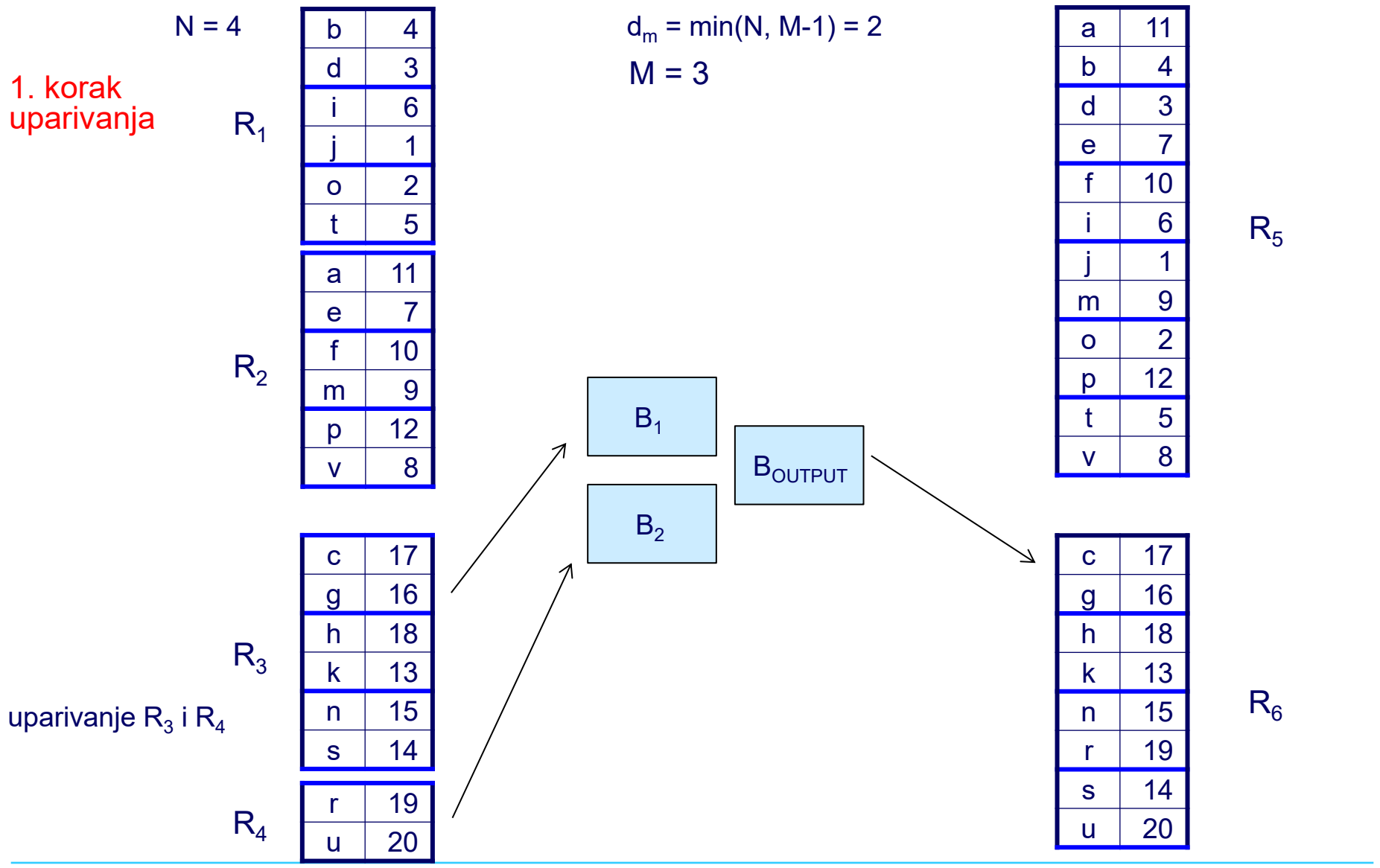
```
read first block of each run Ri into main memory block Bi
repeat
   choose the first tuple (in sort order) among all Bi
   write the tuple to the output* and delete it from Bi
   if Bi is empty and not end-of-file(Ri)
      read the next block of Ri into Bi
until all blocks Bi are empty
```

* *write the tuple to the output*: zapisivanje u postojanu memoriju (uz korištenje međuspremnika) ili prosljeđivanje na ulaz u sljedeću operaciju (*pipelining*)

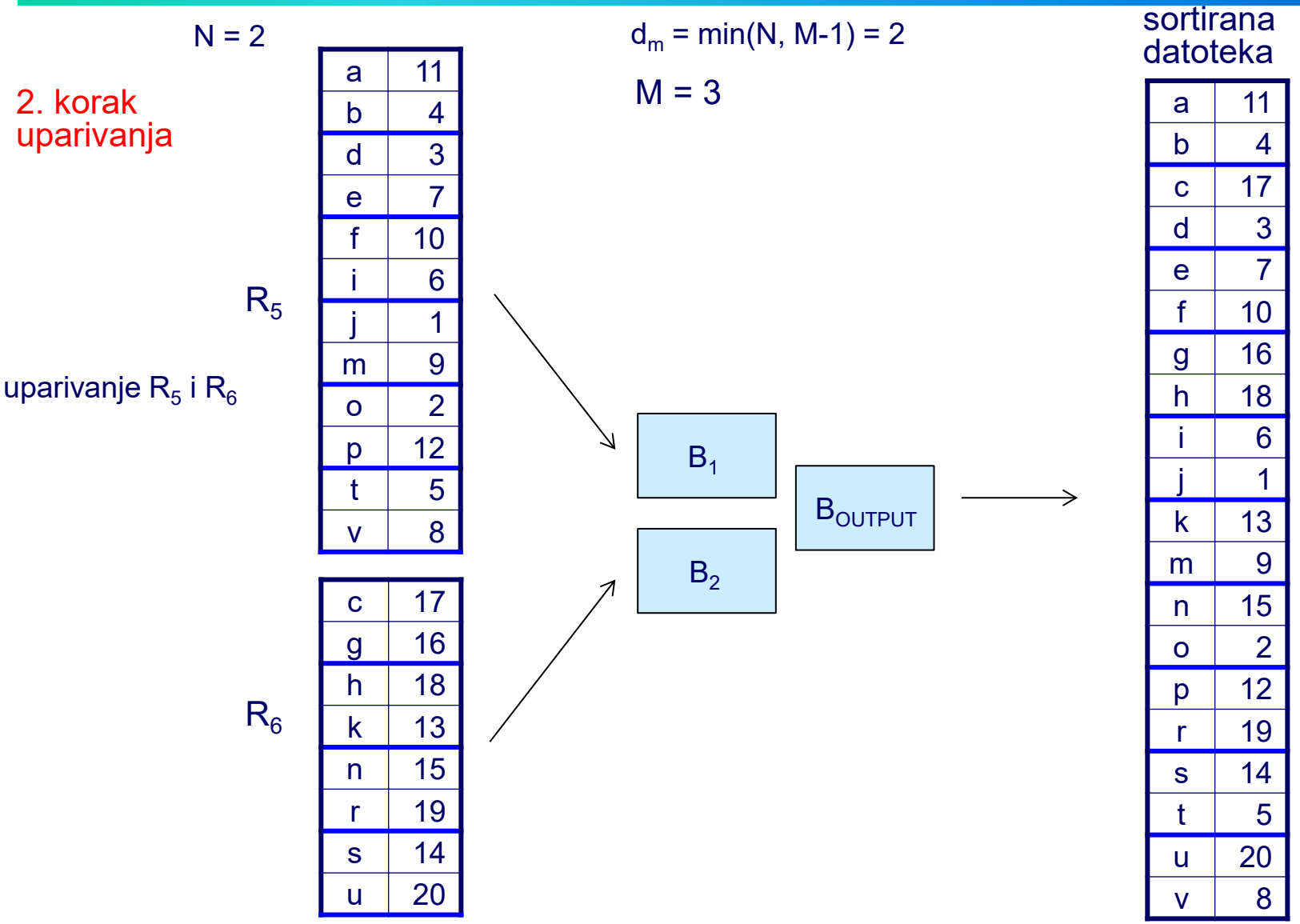
External sort-merge - 2. faza - merging phase



External sort-merge - 2. faza - merging phase



External sort-merge - 2. faza - merging phase



a	11
b	4
d	3
e	7
f	10
i	6
j	1
m	9
o	2
p	12
t	5
v	8

c	17
g	16
h	18
k	13
n	15
r	19
s	14
u	20

a	11
b	4
c	17
d	3
e	7
f	10
g	16
h	18
i	6
j	1
k	13
m	9
n	15
o	2
p	12
r	19
s	14
t	5
u	20
v	8

External sort-merge - procjena troškova

- Zbog prikazanog primjera s ekstremno malom raspoloživom primarnom memorijom, čini se da je ova metoda sortiranja vrlo skupa, međutim u realnim situacijama je vrlo efikasna.

- Primjer
- Sortiranje relacije od 1 000 000 blokova (npr. blok od 2K, ukupno 2GB), pri čemu je na raspolaganju samo 1000+1 blok primarne memorije (približno 2MB).
- U postupku će nastati 1 000 sortiranih segmenata, za koje će biti potreban samo jedan korak uparivanja
  - to znači da se sa samo 2MB primarne memorije, 2GB podataka može sortirati uz utrošak od samo 4 000 000 U/I operacija
  - (to je onoliko U/I operacija koliko je potrebno da bi se datoteka dva puta kopirala iz jednog na drugo mjesto u sekundarnoj memoriji)

External sort-merge - procjena troškova

- u svakom koraku 2. faze broj segmenata se smanjuje za faktor M - 1
- nakon 1. koraka 2. faze rezultat je $\lceil B(r) / M \rceil / (M-1)$ segmenata
- nakon 2. koraka 2. faze rezultat je $\lceil B(r) / M \rceil / (M-1)^2$ segmenata
- nakon i-tog koraka 2. faze rezultat je $\lceil B(r) / M \rceil / (M-1)^i$ segmenata
- nakon zadnjeg koraka (za i=Z) rezultat mora biti ≤ 1 segmenata
  - $\lceil B(r) / M \rceil / (M-1)^Z \leq 1$
- iz toga proizlazi da je Z, tj. broj potrebnih koraka u 2. fazi: $\lceil \log_{M-1} \lceil B(r) / M \rceil \rceil$
- svaki korak druge faze (uparivanje) zahtijeva 2 B(r) U/I operacija
- ukupan broj U/I operacija za 2. fazu: $2\, B(r) \lceil \log_{M-1} \lceil B(r) / M \rceil \rceil$
- broj U/I operacija za 1. fazu (sortiranje): 2 B(r)
  - B(r) *input* + B(r) *output* operacija
- ukupan broj potrebnih U/I operacija: $2\, B(r) + 2\, B(r) \lceil \log_{M-1} \lceil B(r) / M \rceil \rceil$
  - uz uključen trošak zapisivanja konačnog rezultata, inače za B(r) manje
- u primjeru za M=3: broj potrebnih U/I operacija = 60 (uz uračunati trošak zapisivanja rezultata)

Spajanje

1. spajanje s ugniježđenim petljama: *nested-loop join*
2. blokovsko spajanje s ugniježđenim petljama: *block nested-loop join*
3. indeksirano spajanje s ugniježđenim petljama: *indexed nested-loop join*
4. spajanje uparivanjem sortiranih relacija: *sort-merge join*
5. spajanje raspršenim adresiranjem: *hash join*

Nested-loop join

- par ugniježdenih petlji s unaprijed poznatim brojem ponavljanja

$r \bowtie_F s$

```
for each t_r in r
   for each t_s in s
      if F(t_r, t_s)
         add t_r·t_s to the result
```

vanjska relacija (*outer relation*)

unutarnja relacija (*inner relation*)

- veliki trošak (uspoređuje se N(r) · N(s) n-torki)
- najlošiji slučaj: samo po jedan blok svake relacije stanu u primarnu memoriju
  - procjena troška: N(r) · B(s) + B(r) (nisu uključeni troškovi zapisivanja rezultata)
- najbolji slučaj: obje relacije ili samo manja relacija (manja relacija se tada koristi kao unutarnja) stanu u primarnu memoriju
  - procjena troška: B(r) + B(s) (nisu uključeni troškovi zapisivanja rezultata)

$t_r \cdot t_s$: n-torka dobivena ulančavanjem n-torki t_r i t_s

N(r), **N(s)** : broj n-torki u relacijama r i s

B(r), **B(s)**: broj blokova u relacijama r i s

Block nested-loop join

```
for each block B_r in r
   for each block B_s in s
      for each t_r in B_r
         for each t_s in B_s
            if F(t_r, t_s)
               add t_r·t_s to the result
```

- najlošiji slučaj: samo po jedan blok svake relacije stanu u primarnu memoriju
  - procjena troška: B(r) · B(s) + B(r) (nisu uključeni troškovi zapisivanja rezultata)
- najbolji slučaj: (jednako kao u *nested-loop join*)
  - procjena troška: B(r) + B(s) (nisu uključeni troškovi zapisivanja rezultata)

Ako je za operaciju spajanja na raspolaganju primarna memorija veličine M blokova

- 1 blok za čitanje unutarnje relacije, 1 blok za izlazni međuspremnik
- vanjska petlja: umjesto po jedan blok, $\lceil B(r) / (M - 2) \rceil$ puta učitava po M-2 blokova
- procjena troška: $\lceil B(r) / (M - 2) \rceil \cdot B(s) + B(r)$
  (nisu uključeni troškovi zapisivanja rezultata)

Indexed nested-loop join

- preduvjet: na raspolaganju je indeks nad unutarnjom relacijom za atribute prema kojima se obavlja spajanje
- za svaku t_r iz r, pomoću indeksa se pronalaze odgovarajuće n-torke iz s

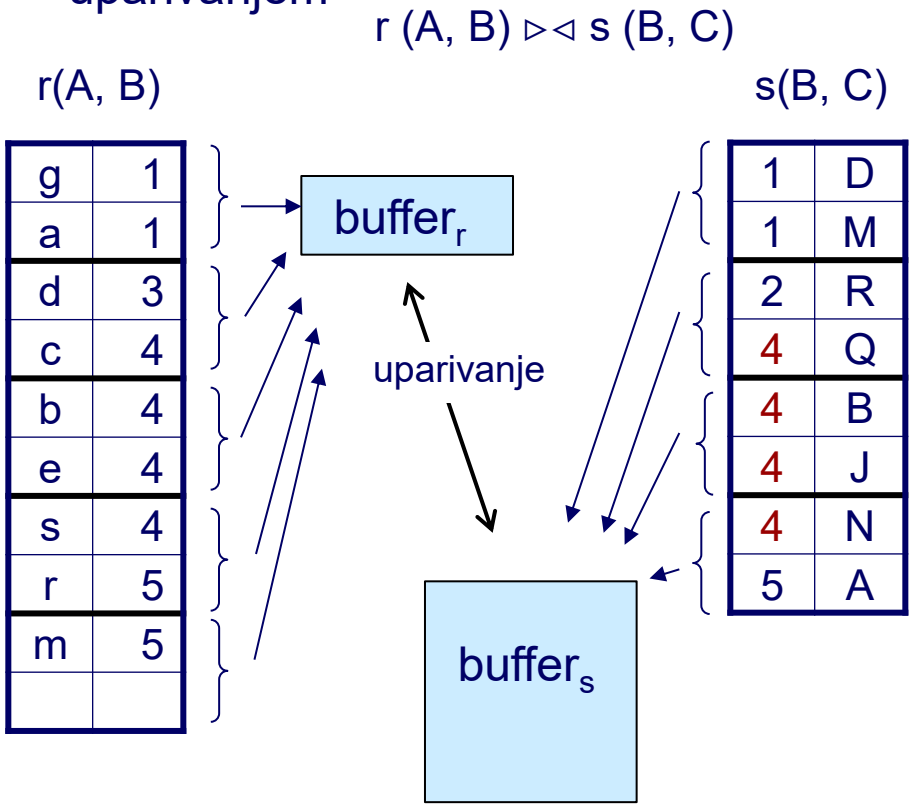
- procjena troška: $N(r) \cdot c + B(r)$ (nisu uključeni troškovi zapisivanja rezultata)

  c : broj U/I operacija za dohvat n-torke iz s pomoću indeksa (~ dubina stabla)

- što ako obje relacije imaju indeksirane atribute prema kojima se obavlja spajanje?
  - u općem slučaju bolje je za vanjsku relaciju odabrati onu s manje n-torki

Sort-merge join

- može se koristiti za prirodno spajanje i spajanje uz uvjet s izjednačavanjem (*equi-join*)
- r i s su sortirane prema atributima prema kojima se obavlja spajanje
- postupak spajanja je sličan drugoj fazi u algoritmu za vanjsko sortiranje s uparivanjem



- pretpostavka: u međuspremnik relacije **s** moguće je odjednom pohraniti sve blokove koji sadrže n-torke s jednakim vrijednostima atributa prema kojima se obavlja spajanje. U primjeru na slici: drugi, treći i četvrti blok zbog vrijednosti 4
- međuspremnik relacije **r** može biti veličine jednog ili više blokova
- moguće korisna nuspojava:
  - rezultat je sortiran prema atributima prema kojima se obavlja spajanje

Sort-merge join

- procjena troška:
  - B(r) + B(s) (nisu uključeni troškovi zapisivanja rezultata)
  - ako relacije nisu unaprijed sortirane, dodati i broj U/I operacija potrebnih za sortiranje relacija r i s, koji ovisi o veličini relacija u odnosu na raspoloživu primarnu memoriju (vidjeti npr. trošak za *external sort-merge*)

Hash join

- može se koristiti za prirodno spajanje i spajanje uz uvjet s izjednačavanjem (equi-join)

  $r\ (A, B) \triangleright\!\triangleleft s\ (B, C)$

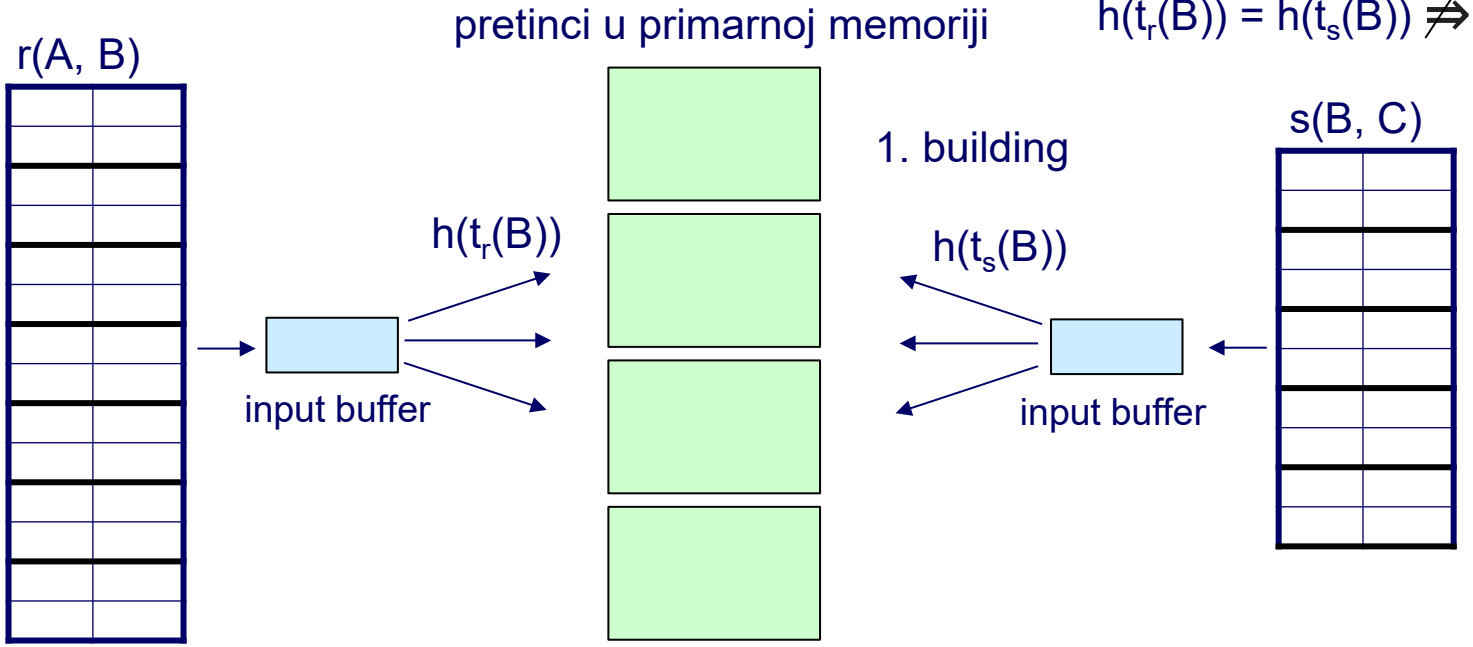
- funkcija raspršenja h preslikava B-vrijednost n-torke u adresu pretinca (*bucket*)
  - adresa pretinca = h(t(B))
  - B može biti jedan atribut ili skup atributa

Hash join

- ako se relacija **s** (manja u paru) može odjednom pohraniti u primarnoj memoriji

- *building phase*
  - n-torke t_s relacije s raspršuju se u pretince u primarnoj memoriji, adresa = $h(t_s(B))$
- *probing phase*
  - za svaku n-torku t_r relacije r izračunava se adresa pretinca $h(t_r(B))$
  - za svaku t_s iz tog pretinca, ako je $t_s(B) = t_r(B)$, dodaj $t_r \cdot t_s$ u rezultat. Provjera je nužna jer funkcija raspršenja za različite t(B) može izračunati istu adresu pretinca

$h(t_r(B)) = h(t_s(B)) \not\Rightarrow t_r(B) = t_s(B)$

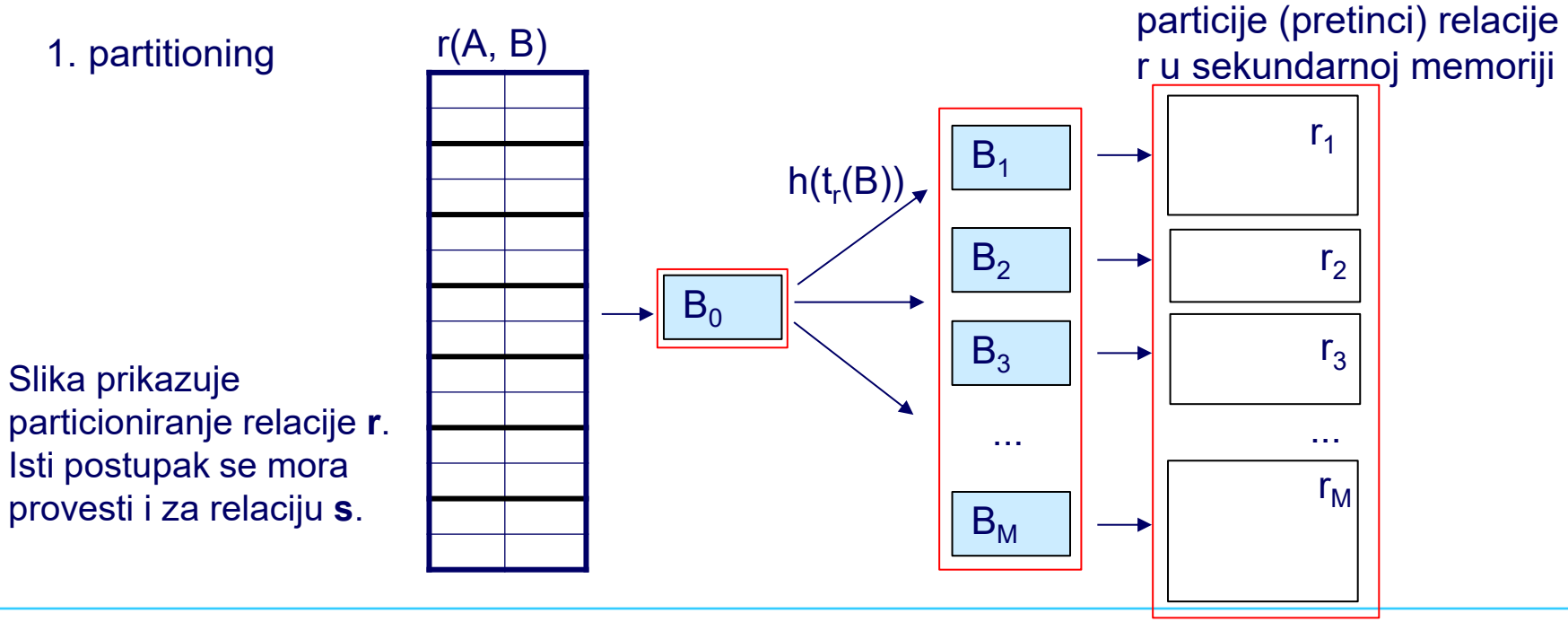


- procjena troška: B(r) + B(s) (nisu uključeni troškovi zapisivanja rezultata)

Hash join

- ako se relacija **s** (manja u paru) ne može odjednom pohraniti u primarnoj memoriji

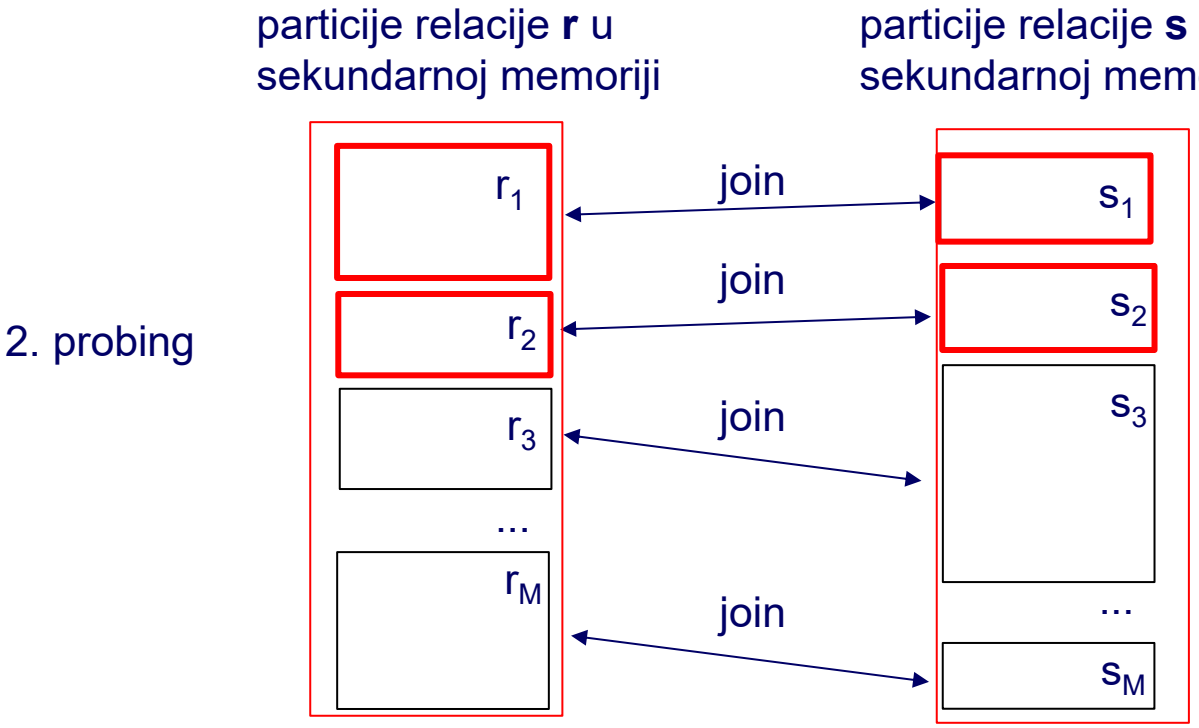
- ako se relacija **s** ne može smjestiti u primarnu memoriju ⇒ *partitioned hash join*
- ***partitioning phase*:** primjenom funkcije raspršenja h(t(B)), zasebno se particioniraju relacije r i s. U sekundarnoj memoriji nastaju particije (pretinci):
  - $r_1, r_2, \ldots, r_M$ i $s_1, s_2, \ldots, s_M$
  - kreiranja M particija zahtijeva najmanje M+1 blok primarne memorije
    - jedan blok (B_0) u funkciji ulaznog međuspremnika (za čitanje relacije), te po jedan blok za svaku particiju (u funkciji izlaznog međuspremnika za i-tu particiju)



Hash join

- ako se relacija **s** (manja u paru) ne može odjednom pohraniti u primarnoj memoriji

- ***probing phase*:** za svaki i = 1..M, obavlja se spajanje n-torki iz particija r_i, s_i
  - npr. primjenom *nested-loop join*, *hash-join* ili *block nested-loop join*



- procjena ukupnog troška: 3 * (B(r) + B(s)) (nisu uključeni troškovi zapisivanja rezultata)
  - za vježbu objasniti zašto. Također, uočiti da će trošak biti veći od navedenog samo u vrlo rijetkim slučajevima kada se pri spajanju nekog para particija niti jedna od particija iz para ne bi mogla odjednom smjestiti u primarnu memoriju.

Usporedba: *Hash join* i *(block) nested-loop join*

- *hash join* u općem slučaju brže evaluira ukupan rezultat
- *nested-loop* brže evaluira "prve n-torke"
  - koristi se ako je važno da korisnik čim prije dobije barem dio rezultata
- *hash join* je efikasniji, ali se može koristiti samo za "*equi-join*" uvjete spajanja

Primjer:

linija

let	udaljenost
CA-825	700
LH-412	4800
BA-722	15000
CA-311	13000

zrakoplov

tip	dolet
B747	13000
A320	5400
DC-9	3100

- sljedeća operacija spajanja se ne može obaviti pomoću raspršenog adresiranja

```
SELECT *
  FROM linija JOIN zrakoplov
         ON dolet >= udaljenost;
```

Vanjsko spajanje

$$r\ (A, B)\ *\underset{F}{\triangleright\!\triangleleft}\ s\ (B, C)$$

- koristi se modificirani oblik jednog od prethodno opisanih algoritama
- npr. modificira se *nested-loop join* (koristi se za vanjsko spajanje uz uvjet)
  - u slučaju lijevog vanjskog spajanja, lijeva relacija mora se koristiti kao vanjska relacija
  - ako se za t_r iz vanjske relacije r ne pronađe odgovarajuća t_s iz unutarnje relacije s, u rezultat se dodaje $t_r \cdot t_{NULL}$ (gdje je t_{NULL} n-torka čije su vrijednosti atributa NULL)
- prirodno vanjsko spajanje (*natural outer join*) ili vanjsko spajanje s izjednačavanjem (*outer join with equi-join condition*) se (osim pomoću *nested-loop*) također mogu realizirati modificiranim oblicima *hash-join ili sort-merge join* algoritama. Za taj oblik spajanja se gotovo uvijek koristi upravo modificirani oblik algoritma *hash-join* (jer je efikasniji od *nested-loop*).

Eliminacija duplikata

δ (r) (ako se relacija r ne može smjestiti u primarnu memoriju)

- pomoću raspršenog adresiranja (najčešće)
  - primjenom funkcije raspršenja h(t) nastanu particije r_1, r_2, ..., r_M (u sek. mem.)
  - za svaku particiju r_i provodi se postupak:
    - formira se nova prazna *in-memory hash table*, pri čemu funkcija raspršenja mora biti drugačija od h (jasno je zašto)
    - čita se particija r_i i za svaku n-torku t_r iz r_i
      - t_r se dodaje u *hash table* samo ako se već ne nalazi u pripadnom pretincu
    - n-torke iz *in-memory hash table* se dodaju u rezultat
  - procjena troška: 3 * B(r) (nisu uključeni troškovi zapisivanja rezultata)

- pomoću vanjskog sortiranja s uparivanjem (*external sort-merge*)
  - već za vrijeme faze sortiranja, eliminiraju se duplikati pronađeni unutar istog segmenta (smanjuje se inicijalna veličina segmenata)
  - preostali duplikati se jednostavno eliminiraju tijekom faze uparivanja
  - procjena troška: jednak kao kod sortiranja

Projekcija

$\pi_L(r) = \delta(\pi^B_L(r))$ (ako se relacija r ne može smjestiti u primarnu memoriju)

- *bag* verzija projekcije (izdvajanje atributa) je jednostavna
- za eliminaciju duplikata koristi se jedan od opisanih postupaka (*hash, external sort-merge*)
- procjena troška: jednaka kao kod odabrane operacije eliminacije duplikata

Agregacija i grupiranje

$A_1, A_2, ..., A_m \mathcal{G}_{\mathcal{AF}1(B_1), \mathcal{AF}2(B_2), ..., \mathcal{AF}n(B_n)} (r)$

Ako se <u>rezultat</u> može pohraniti u raspoloživi prostor primarne memorije

- rezultat se pohranjuje u *in-memory hash table*
- za svaku n-torku s ključem $A_1, A_2, ..., A_m$ izračunava se $h(A_1, A_2, ..., A_m)$
  - ako se još ne nalazi u odgovarajućem pretincu u *in-memory hash table*
    - u pretinac dodati n-torku $A_1, A_2, ..., A_m, \mathcal{AF}_1, \mathcal{AF}_2,..., \mathcal{AF}_n$
  - izračunati i u pretinac natrag upisati nove vrijednosti $\mathcal{AF}_1, \mathcal{AF}_2,..., \mathcal{AF}_n$
- procjena troška: B(r) (nisu uključeni troškovi zapisivanja rezultata)

Ako je <u>rezultat</u> prevelik za raspoloživi prostor primarne memorije

- formirati particije $r_1, r_2, ..., r_M$ u sekundarnoj memoriji, kao kod eliminacije duplikata raspršenim adresiranjem
  - za svaku particiju zasebno provodi se gore opisani postupak
  - procjena troška: 3 * B(r) (nisu uključeni troškovi zapisivanja rezultata)

Unija, presjek, razlika

(ako se manja relacija ne može smjestiti u primarnu memoriju)

- **pomoću vanjskog sortiranja s uparivanjem**
  - obje relacije sortirati po svim atributima
  - tijekom uparivanja

∪
- n-torku iz **r** ili **s** dodati u rezultat, preskočiti sve jednake n-torke u obje relacije

∩
- n-torku iz **r** koja se nalazi i u **s** dodati u rezultat, preskočiti sve jednake n-torke u obje relacije

\
- n-torku koja se nalazi u **r**, a ne nalazi se u **s**, dodati u rezultat, preskočiti sve jednake n-torke u obje relacije

- procjena troška:
  - ako su r i s već prije sortirane
    - B(r) + B(s)
  - ako r i s nisu sortirane
    - trošak sortiranja (r) + trošak sortiranja (s) + B(r) + B(s)

Unija, presjek, razlika

(ako se manja relacija ne može smjestiti u primarnu memoriju)

- **pomoću raspršenog adresiranja**
  - primjenom funkcije raspršenja h(t) nastanu particije
    - $r_1, r_2, ..., r_M, s_1, s_2, ..., s_M$
  - ponavljati za svaki r_i
    - $r_i \rightarrow$ *in-memory hash table*

$\cup$
- za svaku n-torku t_s iz s_i koja se već ne nalazi u *in-memory hash table*
  - dodati t_s u *in-memory hash table*
- sadržaj *in-memory hash table* dodati u konačni rezultat

$\cap$
- za svaku n-torku t_s iz s_i koja se nalazi u *in-memory hash table*
  - dodati t_s u rezultat

\
- za svaku n-torku t_s iz s_i koja se nalazi u *in-memory hash table*
  - izbaciti t_s iz *in-memory hash table*
- sadržaj *in-memory hash table* dodati u konačni rezultat

Literatura:

- H. Garcia-Molina, J. Ullman, J. Widom. Database system implementation. Prentice-Hall International, Inc. Upper Saddle River, New Jersey. 2000.
- A. Silberschatz, H. Korth, S. Sudarshan. Database system concepts, 6th ed. McGraw-Hill. 2010.
- R. Elmasri, S. Navathe. Fundamentals of database systems, 3rd ed. Addison-Wesley Publishing Company. Reading, Massachusetts. 2000.
- IBM Informix Dynamic Server Performance Guide, Version 11.50, IBM, 2008.